Friedrich-Alexander-Universität
Faculty of Engineering

Optimization of Photovoltaic and Battery Storage Systems

Christian Besold, Jakob Haberstock, Nandini Joshi, Robert Leichtl, Florian Merlau

Friedrich-Alexander-Universität Erlangen-Nürnberg, Computer Science 7

July 30, 2025

Abstract

This report examines how homeowners can optimize the amount of Photovoltaik (PV) and battery modules to maximize cost savings.To help achieve this goal, a simulation model was developed in AnyLogic. The model features user-configurable inputs, enabling tailored simulations for individual households. Results from an eight-year run indicate that investing in solar panels is highly likely to yield long-term financial benefits. However, under the cost assumptions used in this study, battery storage systems were not found to be economically viable due to their high upfront costs outweighing the potential monetary savings.

Contents

List of Figures

List of Abbreviations

EV Electric Vehicle. 4, 6, 38, 39, 44

kWh Kilowatt hour. 32, 40

kWp Kilowatt Peak. 10, 18, 33, 34, 40, 41

MILP Mixed Integer Linear Program. 23

PV Photovoltaik. 1, 4, 6, 7, 10–13, 16, 18, 19, 22, 23, 33–36, 38, 40–47

SoC State of Charge. 6, 22, 32, 40

1 Introduction

In the wake of the global energy transition and rising electricity prices, decentralized energy resources like residential PV systems and battery storage are gaining significant attention. They offer households a path toward greater energy independence and the potential for significant cost savings. However, realizing these benefits presents a considerable economic and technical challenge. The core difficulty lies in the complexity of the investment decision: the interplay of intermittent solar generation, variable household consumption, and dynamic electricity prices makes it nearly impossible for homeowners to intuitively determine the optimal system size.

This project addresses this exact challenge by developing a comprehensive, simulation-based optimization model. Our research question is: **What is the optimal number of PV and battery modules to maximize the financial benefit for a household?**. To answer this question, our model simulates a multitude of system configurations and operational strategies. It identifies the best combination of PV and battery capacity for a given household profile by integrating crucial real-world factors. These include detailed household consumption patterns, the energy demands of heat pumps and EVs, dynamic electricity tariffs, and location-based PV generation data. The model thus serves as a powerful tool to find the optimal balance between initial investment costs and long-term savings.

1.1 Limitations

While the simulation model provides a comprehensive framework for optimization, it is subject to several inherent limitations that should be considered when interpreting the results.

- **Data Granularity and Interpolation:** The model relies on historical PV production and electricity price data that was originally available in hourly intervals. This data was linearly interpolated to the 15-minute resolution used in the simulation. This process inevitably smooths out the high-frequency volatility characteristic of both solar generation (e.g., due to passing clouds) and real-time electricity markets. Consequently, the model may underestimate the need for battery storage as a buffer against rapid fluctuations. (See Figure 5)
- **Simplified Component Models:** The physical components are represented by simplified models. The PV production calculation assumes optimal, fixed installation parameters (azimuth and angle) and employs a linear degradation factor. The battery model uses a constant round-trip efficiency and a cycle-based degradation model, omitting complex real-world factors like the impact of temperature, State of Charge (SoC), and discharge rate on performance and lifespan.
- **Static Load Profiles:** The model utilizes pre-defined, static load profiles for household consumption, EV charging, and heat pump operation. In reality, user behavior is stochastic and can vary significantly from day to day. The model does not account for dynamic demand-side management, where consumers might alter their consumption patterns in response to price signals or energy availability.
- **Economic Assumptions:** The economic analysis is based on initial investment costs for PV modules and batteries. It does not factor in potential future costs such as maintenance, inverter replacement, or repairs. Furthermore, it assumes the current electricity tariff structure remains unchanged over the entire simulation period.

1.2 Demarcation

To ensure a focused and feasible analysis, the scope of this project was deliberately defined with clear boundaries. This work is an economic optimization study for a single residential entity and should not be misconstrued as a broader grid-level analysis. The following points outline the key demarcations of our model:

- **System Scope:** The analysis is strictly confined to a single-household system. It does not model interactions with other prosumers, participation in local energy communities, or the cumulative impact of widespread decentralized generation on the stability and operation of the public electricity grid (e.g., voltage rise or congestion issues).
- **Optimization Strategy:** The optimization process selects the best-performing strategy from a **predefined set of trading rules**. The model does not develop or learn novel, adaptive control strategies, for instance through reinforcement learning or advanced forecasting methods.

2 Project Definition

This project aims to address the economic challenge homeowners face when investing in renewable energy systems. The central goal is to determine the optimal configuration of PV modules and battery storage to maximize long-term financial benefits. To achieve this, a sophisticated simulation model was developed to serve as a decision-support tool.

The project is defined by the following key aspects:

1. **Central Research Question:** The core of this work is to answer the question: **What is the optimal number of PV and battery modules to maximize the financial benefit for a household?** The model seeks to find the ideal balance between the upfront investment costs and the operational savings realized over an extended period.
2. **Scope and Boundaries:** The analysis is intentionally focused to ensure clear and relevant results:
    - **System Scope:** The model analyzes a **single residential household**. It does not consider grid-level impacts, energy communities, or peer-to-peer trading.
    - **Optimization Approach:** The simulation identifies the best configuration by testing a **predefined set of control strategies** (e.g., a simple "Greedy" strategy versus an "Optimizer-based" strategy). It does not involve developing new control algorithms through methods like machine learning.
    - **Economic and Regulatory Framework:** The simulation operates using historical electricity price data and current cost structures. It does not forecast future changes in policy, subsidies, or market design.

3. **The Simulation Model: Technical Implementation** To answer the research question, a detailed agent-based simulation was built using **AnyLogic**. The model's architecture is modular, with distinct components representing different parts of the household energy system:

    - **Agents and Java Classes:** The system is composed of several agents (e.g., `PV`, `Battery`, `Heat Pump`, `Household Consumption`). The core logic for each agent is implemented in corresponding **Java classes** (e.g., `PVJava`, `BatteryJava`). The AnyLogic agents primarily serve as communication hubs, using their ports to send and receive messages, while the heavy lifting (calculations, etc.) is handled by the Java objects they contain.
    - **System Orchestration:** The `House Controller` acts as the central brain of the simulation. In each 15-minute time step, it orchestrates the flow of information. This will be described in more detail in Section 4.1.

      This agent-based structure allows for a clear separation of concerns and a scalable model where new components (like the `Electric Vehicle`) can be integrated with relative ease. The use of dedicated Java classes for the logic ensures robust and efficient computation throughout the simulation run.

2.1 Requirements Definition & Detailed Specification

2.1.1 Requirements Definition

2.1.1.1 Functional Requirements

- **Simulation of Energy Flows:** The model must simulate the energy flow between the PV system, household, battery storage, heat pump, electric vehicle, and the public grid in discrete 15-minute time steps.
- **Parametrization:** The user must be able to define key input parameters such as the number of PV modules, battery capacity, budget, location, and the number of people in the household.
- **Economic Calculation:** The system must calculate the total costs (investment) and the operational savings over the simulation period to evaluate profitability.
- **Strategy Comparison:** The model must implement and compare at least two control strategies: a simple, rule-based "Greedy" strategy and a predictive, optimizer-based strategy.

2.1.2 Detailed Specification

- **System Architecture:** The simulation is implemented as an agent-based model in `AnyLogic`. A central `House Controller` agent coordinates the interactions between subordinate agents representing individual physical components (`PV`, `Battery`, `HeatPump`, etc.).
- **Data Flow and Interfaces:** Communication between agents is asynchronous and based on a message-passing system. The `House Controller` sends requests (`DataResponseTriggerMessage`), and the component agents respond with their current data (`DataMessageAnswerCurrent`).
- **Data Model:**
  - **Input:** The input data for load profiles, PV generation, and prices are provided as the "init.csv" file (see appendix). At startup, these are imported into a `PostgreSQL` database and loaded into an in-memory cache (Java `HashMap`) for fast access during the simulation.
  - **Output:** The primary outputs of the simulation are economic performance indicators (e.g., `moneySaved`, `valueGain`), which are displayed on the `AnyLogic` UI or saved to a results file.
- **Simulation Core:**
  - **Time Model:** The simulation proceeds in fixed 15-minute intervals.
  - **Optimization Module:** For the optimized strategy, an external solver (`Gurobi`) is called via a defined interface, to which forecast data for a specific horizon is passed.

2.2 Project Planning and Timeline

Figure 1 illustrates the project schedule and key milestones using a Gantt chart.

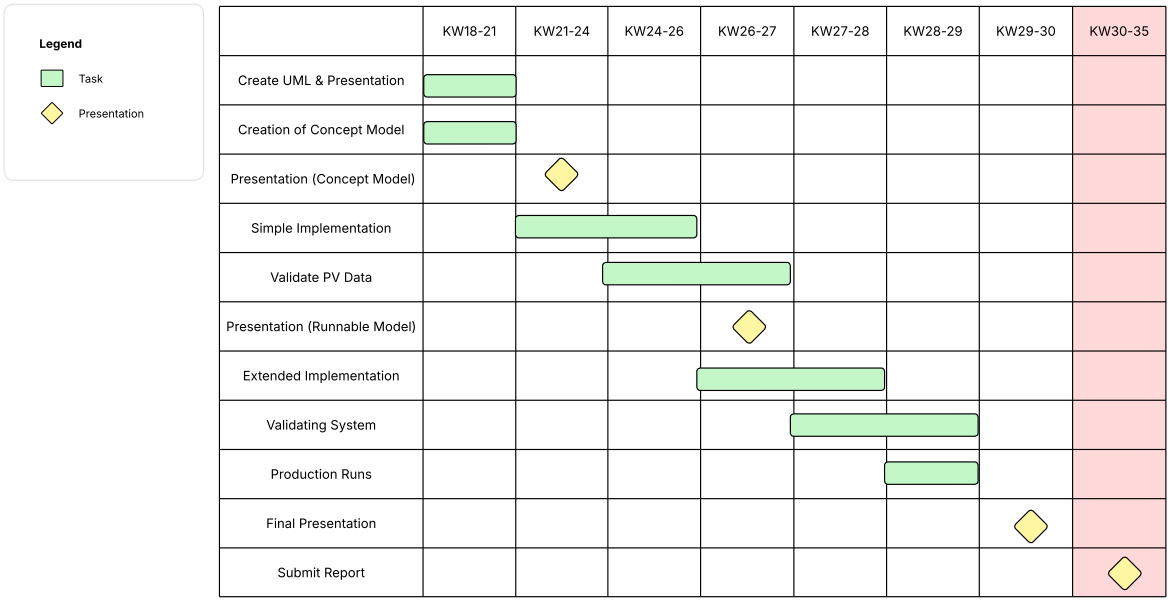

Figure 1: Project Gantt Chart

The full team planning and the complete Gantt chart can be found in our project repository [3].

3 Input Modeling

3.1 Data Collection

Data collection for the simulation was a multi-stage process, as specific, high-quality time-series data was required for each component of the energy system, from generation to consumption. The **House Controller**, as the central control unit, relies on this data to optimally manage the energy flow within the household. A key challenge was sourcing data from various origins and normalizing it to a uniform 15-minute interval.

3.1.1 Photovoltaic Production Data

There were two possible approaches to achieve the simulation of PV modules.
Firstly, many different real-world factors for PV production, such as temperature, cloud cover, solar radiation, and pollution data could be used. Those different factors would then be molded together to calculate a new PV production dataset, which then could be compared to real world datasets to validate it. This approach would incorporate more factors and, if implemented correctly, would be more realistic than the second approach, but was deemed unfitting for this project. Multiple systems must be simulated. Therefore, devoting significant effort to the detailed simulation of just a single component is not feasible.
The second approach was to search for real-world datasets. The problem here was that all available datasets had no additional information provided, especially the Kilowatt Peak (kWp) was often missing. This attribute was critical for calculating our PV module production, it was needed to scale the dataset results by the ratio of our own total installed kWp to the kWp used in the dataset, to obtain correct production values. This led us to use the Photovoltaic Geographical Information System PVGIS [1].

This interactive EU Science Hub tool provides solar radiation data and also calculates the resulting PV production for given input variables and a given location which can be freely chosen (see Figure 2 and Figure 3). The variables used for the Hourly Data sections on the given website were the following:

- start and end year: 2005 - 2023
- Solar radiation database: PVGIS-ERA5
- Mounting type: fixed
- Optimize slope and azimuth
- PV technology: Crystalline silicon
- Installed peak PV power [kWp]: 1
- System loss [%]: 14
- PV Power and Radiation Components boxes both checked
- location: Konstanz

Konstanz was chosen as the household location, due to real world personal consumption data for the city being available. By using the same location for both datasets, it was ensured that local patterns are considered.

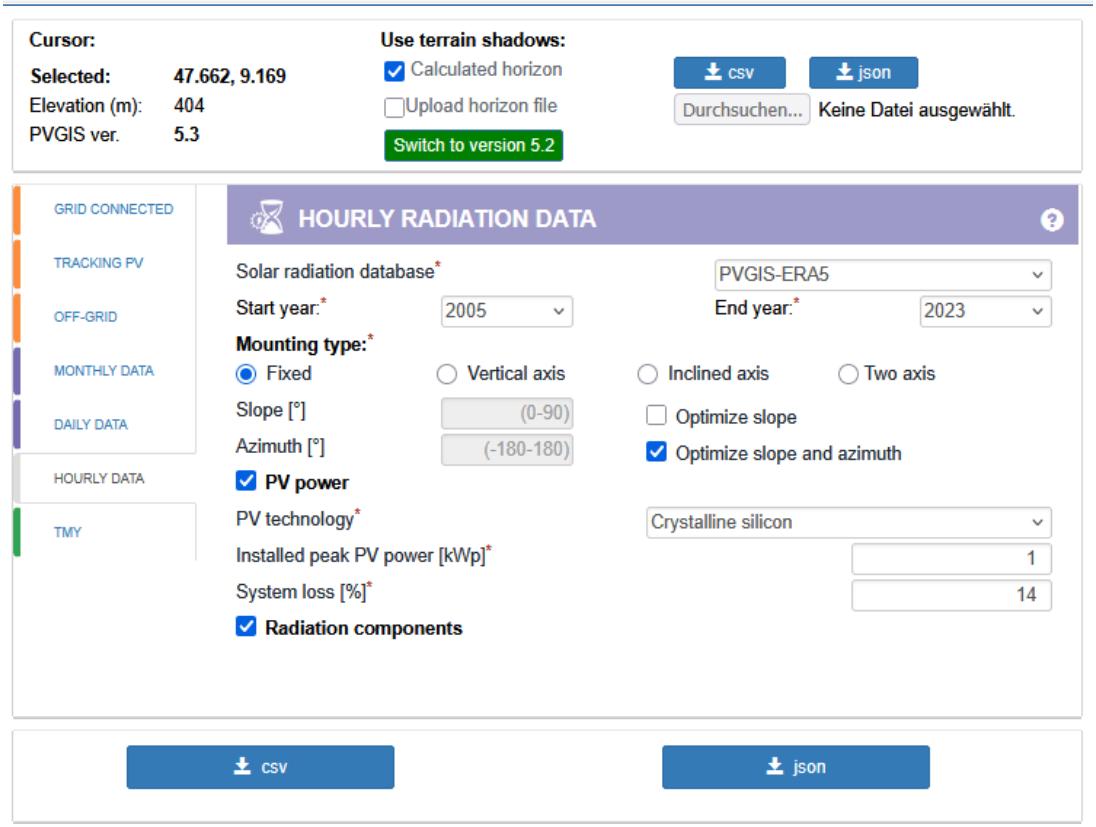

Figure 2: PVGIS interface with correctly selected inputs

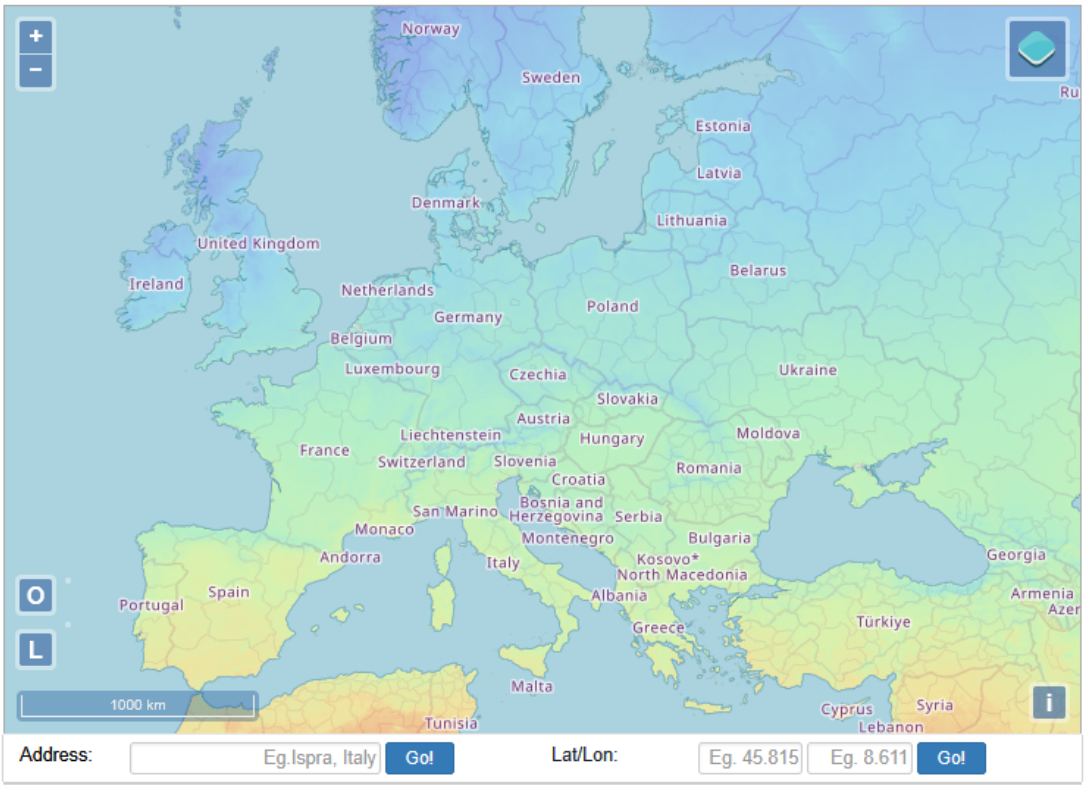

Figure 3: PVGIS location selection interface

Those inputs resulted in the renamed file PVRawData.csv which included a header and unnecessary columns for the solar radiation and other data. The time interval for this initial csv file was 1 hour.
After converting the file to a more useable format (see 3.2) and completing the validation of the provided data (see 3.3), this dataset could then be used as the foundation for further PV calculations.
Figure 5 shows three randomly selected days with their corresponding PV and price diagrams.
Below is a small excerpt of the resulting csv file for clarification, kWh is the amount of energy produced in the current 15 minute interval and time is the starting time step of the interval.
Note that the time format used in Figures 4 and 5, as well as in our dataset in general, is UTC time.

Time	kWh
2005-01-01 06:30:00	0.0
2005-01-01 06:45:00	6.25e-07
2005-01-01 07:00:00	1.25e-06
2005-01-01 07:15:00	1.875e-06
2005-01-01 07:30:00	2.5e-06
2005-01-01 07:45:00	0.0011

Figure 4: Converted PV.csv file

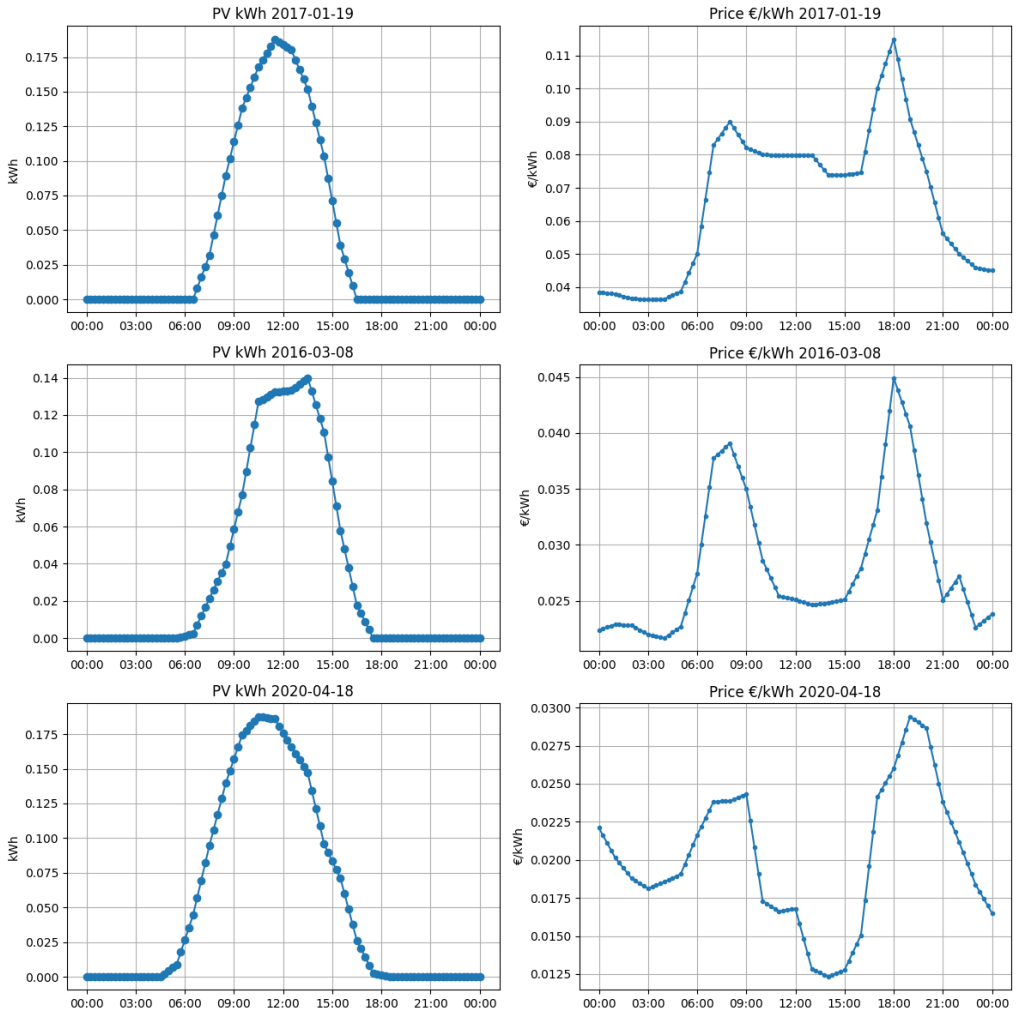

Figure 5: PV production and prices plotted for 3 random days

3.1.2 Electricity Price Data

Typical households buy their electricity at stable prices for 12 or 24 months. This approach would not suit us because our optimizer needs fluctuating energy prices to work properly.
To solve this problem, wholesale prices, which are normally only accessible to companies, were considered. After searching for a while, a solution to this problem was found. Private households can access wholesale prices via an Austrian company called awattar. They offer an hourly contract based on wholesale prices [1].
For more information on this contract and its implementation, see 4.2.6.
It is important to note that in our simulation, this contract is used for both selling and buying energy. In reality, the selling side of household energy is different. Typical households can only sell their own solar production and get a fixed sum per kWh to sell solar energy. This so-called Netzeinspeisevergütung is about 8 cent per kWh sold and typically gets lower over time.
Now, only the wholesale prices where needed, as a basis for our calculation. The source for this data was the so-called German government agency Bundesnetzagentur. They have a section to download data on their website[2].

[1] https://www.awattar.at/tariffs/hourly
[2] https://www.smard.de/home/downloadcenter/download-marktdaten/

The variables used for the wholesale prices on the given website were the following:

- Main category: Market
- Data category: Day-ahead prices
- Country: Germany
- 1.1.2016 - 31.12.2024
- Resolution: Hour
- Filetype: CSV

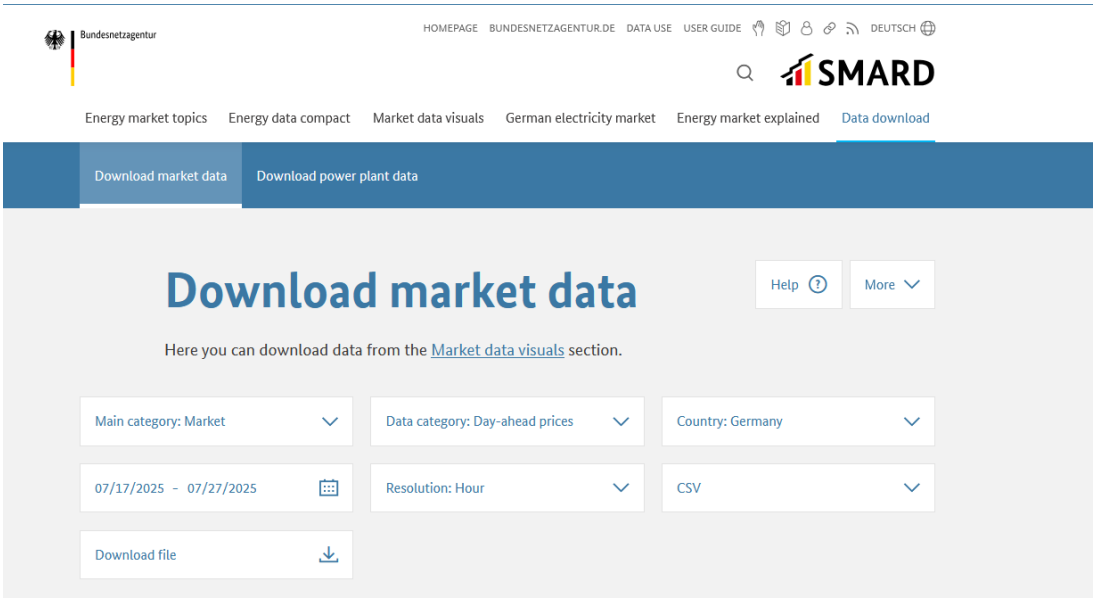

Figure 6: Bundesnetzagentur interface with correctly selected inputs

Similarly to the PV data, there was also the need to clear unnecessary headers and columns, which was done with the same program. The only difference here was that due to a change in nation-wide energy market structures, the dataset had 2 distinct columns that were useful. There was not a single column that had the whole data, but instead two columns, which only had data for half of the time span. After identifying this issue, every other column was omitted, and only the two useful columns were combined, resulting in our final hourly dataset.
Despite there being a quarter-hourly option on the website, it only copied the price of the hourly dataset to three more data points per hour. To achieve greater variability in prices, the data was interpolated manually.

See Section 3.2 for how the hourly intervals were converted to the desired 15-minute intervals.

See Figure 5 for a visualization of three randomly selected days of price data on the right.

Below is a table consisting of a small excerpt of the final price.csv file for visualization purposes.

Time	Price (Euro/kWh)
2024-09-10 19:45:00	0.0856
2024-09-10 20:00:00	0.0832
2024-09-10 20:15:00	0.0829
2024-09-10 20:30:00	0.0826
2024-09-10 20:45:00	0.0823
2024-09-10 21:00:00	0.0820
2024-09-10 21:15:00	0.0806

Figure 7: Converted price.csv file

3.1.3 Heat Pump Data

Acquiring consumption data for the heat pump posed a challenge. We did find electric consumption data in the internet but the problem was, that we had no information at all about the house the data is from. If we used that dataset we wouldn't know if it fits to our household data for example. To address this, we used heat pump data provided by Jonathan from his proprietary model. He generated data covering a 10-year operational period for a specific scenario: a 130-square-meter house with a 3-person household. The other parameters are for a typical household in Germany. The file where all default values are defined in the "default_parameters.xlsx" file (see appendix). The original data, which had an hourly resolution, was blown up to 15-minute intervals to align with the simulation requirements.

3.1.4 Household Consumption Data

The dataset used for modelling personal energy consumption originates from the city of Konstanz, Germany, and covers detailed household electricity usage for the full calendar year of 2016. Measurements were captured at 15-minute intervals, resulting in a high-resolution time series suitable for simulation purposes. Importantly, each recorded value represents the total energy consumption of the household from the beginning of the year up to the specified timestamp. This means the dataset does not directly provide interval-based consumption between two consecutive readings, but rather an aggregate figure that continually increases over time. As such, any downstream analysis that requires energy usage within a specific period must compute the difference between two adjacent timestamps.
A critical characteristic of this dataset is its focus on household-level consumption limited to shared appliances. It includes usage data from commonly used electrical devices such as refrigerators, washing machines, dishwashers, and other central domestic appliances. Personal devices like laptops, phone chargers, or room-specific cooling units are not individually tracked in the readings. This makes it difficult to isolate the energy usage of individual occupants, as the dataset lacks any person-specific consumption labels or segmentation. This poses a constraint in accurately modeling individual user behavior, as the dataset does not break down consumption by occupant or room. However, the data does still provide a generalizable approximation of average shared consumption, which can be scaled on a per-person basis using occupancy assumptions.
To approximate total household consumption more realistically, a multiplier of 5 was applied during data processing. This adjustment is based on the estimation that the recorded appliances represent roughly 20% of a full household's total energy use, allowing us to scale the shared appliance data up to reflect complete household-level consumption. This approach assumes linear scaling with the number of occupants (e.g., multiplying the average by 2 for 2 people), which, while not perfect, is necessary given the lack of individualized usage data.

Another key aspect is the dataset's time frame limitation to the year 2016. To enable its use for simulations involving any future dates, a looping mechanism is applied. In this approach, any input timestamp, whether in the past, present, or far future, is transformed into a structurally equivalent timestamp within 2016 by retaining the original month, day, hour, and minute, and replacing only the year. This timestamp conversion is handled automatically by a dedicated class in the simulation. This strategy allows the fixed dataset to be extrapolated across time, enabling predictive modeling and future scenario analysis without requiring access to post-2016 data.
Overall, even though the dataset has some limitations in detail and coverage, its high level of detail and organization make it a strong base for estimating energy use in a shared home.

Below is a section of the data gathered from one house.

utc_timestamp	DE_KN_residential1_dishwasher	DE_KN_residential1_freezer	DE_KN_residential1_washing_machine
2016-01-01 00:00:00Z	100.627	78.109	93.963
2016-01-01 00:15:00Z	100.627	78.126	93.963
2016-01-01 00:30:00Z	100.627	78.126	93.963
2016-01-01 00:45:00Z	100.627	78.144	93.963
2016-01-01 01:00:00Z	100.627	78.144	93.963

Figure 8: Household Load Dataset sample for 1 house

3.2 Data Fitting

A general issue was encountered regarding the differing time intervals of the datasets. A uniform interval size was required across all data sources. The smallest step size where 15 minute intervals for the personal consumption data which led us to using this for all other datasets. The hourly intervals our price and PV datasets used were changed via linear interpolation. For this reason, csv_and_plot.py was developed. This python file not only cleaned up the headers and unnecessary columns but also handled the correct linear interpolation. For the prices, the sum of all values had to quadruple since prices are not time dependent like production. For the PV modules, the sum had to stay the same for the whole dataset, production is measured and the total energy produced should not change between the two interval sizes.
Both constraints were successfully verified with a simple look over the before and after sums of the relevant data.
Another method this file had was plotting 3 random days which were in both datasets to visually validate the correct interpolation and overall dataset.
(See Figure 5 which was produced by the method)

For the household consumption the original dataset provides cumulative household energy consumption values recorded every 15 minutes. To obtain discrete energy usage per interval, the data had to be transformed into a differential format. This was done by computing the difference between consecutive timestamps, where the energy consumed in a 15-minute window is derived by subtracting the previous cumulative value from the current one. This conversion allows the simulation model to analyze energy usage as a time series of discrete intervals, enabling event-based or time-stepped predictions. To adapt these values for modeling individual consumption behavior, each 15-minute interval reading was divided by the number of people assumed to be living in the household. This yielded an estimate of the average energy consumed per person within each interval, assuming uniform usage across all occupants. Although this assumption introduces simplifications, it provides a practical and computationally efficient approximation of per-person consumption in the absence of individualized data. Furthermore, a scaling multiplier was applied to compensate for the partial nature of the dataset.

Since the data includes only a subset of shared household appliances and omits several other significant consumption sources (such as television, entertainment systems, or personal electronics), we introduced a scaling factor of 5. This multiplier is based on the heuristic assumption that the observed shared device consumption represents approximately 20% of the total household load. By scaling the average per-person consumption accordingly, the model better approximates real-world electricity usage patterns in typical domestic environments. This fitted and re-scaled data was then used to generate both current and forecasted consumption values during simulation, allowing the model to dynamically compute energy demand based on user-defined inputs like time range and occupancy.

utc_timestamp	DE_KN_res	DE_KN_res	DE_KN_res	DE_KN_res	DE_KN_res	DE_KN_res	average_per_person_consumption
2016-01-0100:00:00Z	0	0	0	0	0	0	0
2016-01-0100:15:00Z	0.009	0.006	0.005	0.007	0.008	0.028	0.008
2016-01-0100:30:00Z	0	0.012	0.003	0	0.007	0.026	0.006
2016-01-0100:45:00Z	0.009	0.006	0.003	0.004	0.011	0.017	0.006
2016-01-0101:00:00Z	0	0.015	0.003	0.024	0.007	0.022	0.009

Figure 9: Household data with consumption calculated per person

3.3 Validation of Input Models

The only dataset used that had to be validated was the PV data which was not real world data but came from internal calculations which could not be accessed. To adhere to our strict standards, we wanted to make sure that this calculated data was accurate.
Since we aimed to compare our PV dataset to real-world data, a suitable reference first needed to be acquired. The personal consumption dataset from Konstanz included a column for PV production, which was utilized for this purpose. To ensure comparability, the real-world Konstanz data was normalized to match the annual sum of the calculated PV data. This allowed the comparison of both datasets for the same year visually and through specific evaluation metrics. The original calculated data was still used instead of the Konstanz data due to being more flexible with household locality, having a wider time span to access (the personal consumption only had one year of data) and due to knowing the kWp our calculated data uses.

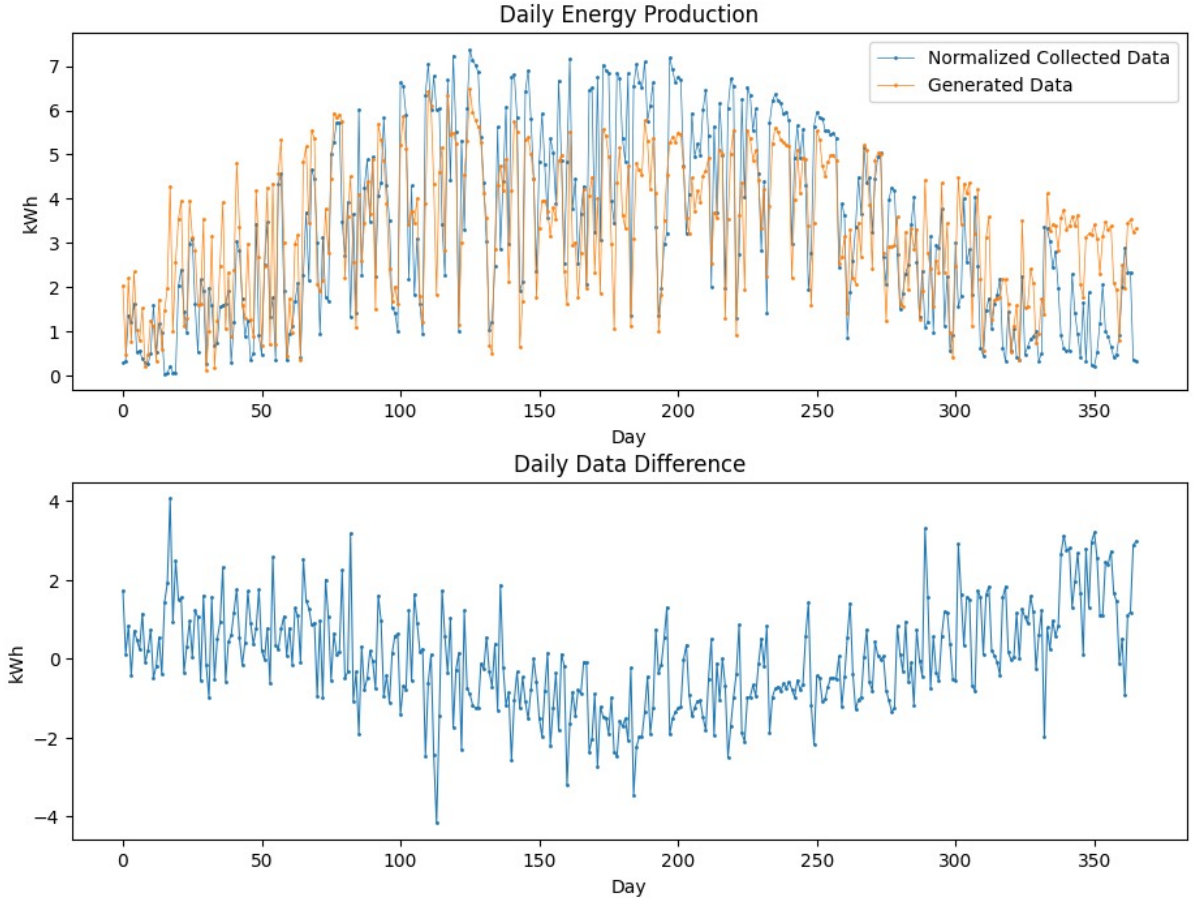

Figure 10: Comparison of real world and calculated PV data

As illustrated in the graphs, the overall match between generated and collected PV data is quite reasonable. Nevertheless, certain discrepancies remain that may influence the reliability of our results. While the general patterns such as peaks and dips align fairly well on most days, the seasonal behavior shows some divergence: the generated data tends to overestimate production in colder months and underrepresents peak values during summer. The second plot further highlights these day-to-day deviations, which fluctuate noticeably throughout the year.

After this brief visual analysis, the following measurements will be discussed:

- MAE: 0.0153
- RMSE: 0.001
- R^2 0.7143
- Pearson r: 0.847

These deviations are not only visually noticeable but also reflected in the performance metrics. The Mean Absolute Error of 0.015 and Root Mean Squared Error of 0.001 indicate that although the day-to-day accuracy is reasonable, some smaller fluctuations persist. The coefficient of determination (R^2) of 0.7143 shows that a significant portion of the variance in the real-world data is captured by the generated data. Additionally, the high Pearson correlation coefficient ($r = 0.847$) underlines a strong linear relationship between both datasets.
Although not being perfect, the fit is good enough for us to use the calculated / generated data. Just keep in mind that the simulation might lack some of the typical seasonality effects of PV modules which could be yet another factor influencing battery cost saving potential.

4 Model Description

4.1 Conceptual Model

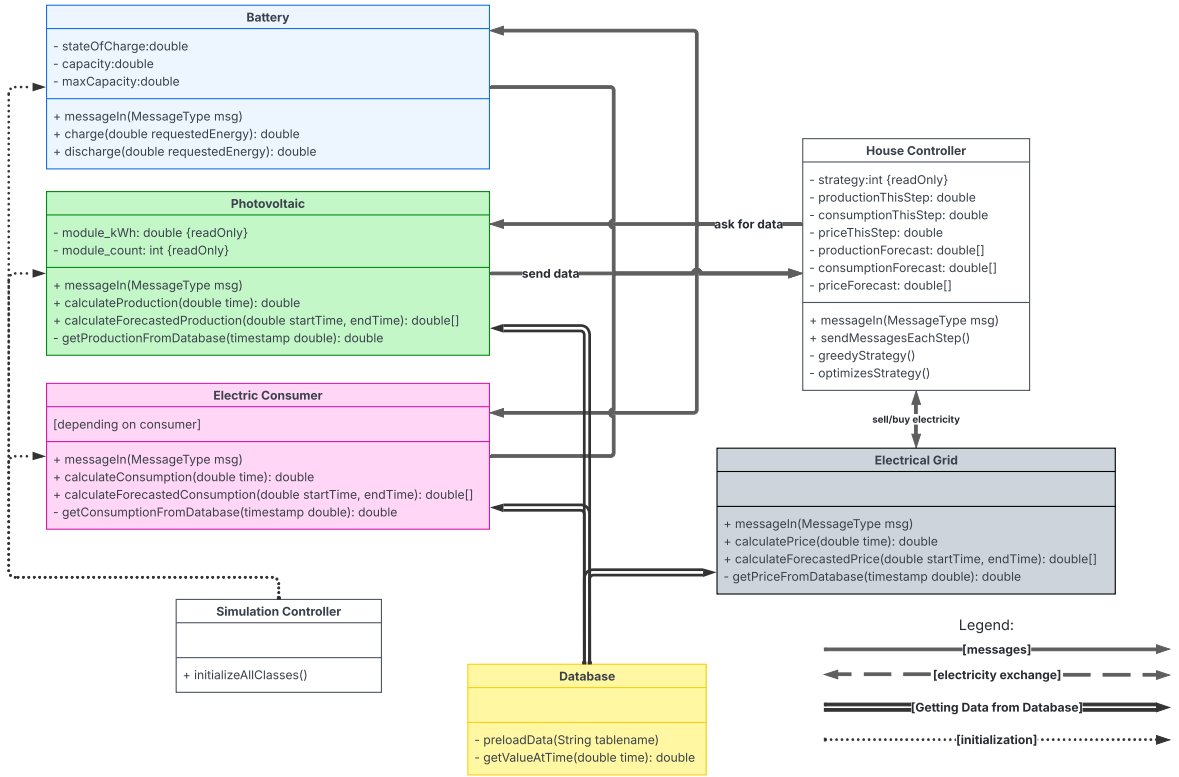

Figure 11: Conceptual Model Diagram

In Figure 11, you can see our Conceptual Model. It does not show all attributes and functions we use. Only those that are important to understand the model. We also use helper classes for type conversions etc.. For simplicity they are also not shown in this Conceptual Model.
All classes between which there are messages sent are agents in AnyLogic. Each of them also has a Java class where all the logic is implemented. So the AnyLogic agent is only there for sending and receiving the messages. If an agent receives a message, it calls the `messageIn` function of the dedicated Java class in the port of the agent. In this `messageIn` function the message is treated depending on the type of the message. We check what kind of message it is using the `instanceof` operator. For more about the different messages types see Section 4.2.1. Additionally, each class has functions to calculate their respective quantities and to get data from the Database. The House Controller also has different functions for the different strategies, but only the function for the active strategy is called.

4.1.1 Sequence of the Simulation

When the simulation is started, the first thing that happens is, that the Simulation Controller initializes all other classes with the parameters which are defined in the "init.csv" file (see appendix).
After that, the simulation starts, and in each simulation step, which is a 15-minute interval, the House Controller sends a message to most other classes. That triggers all other things that happen in each step. An outline of those things that happen each step is presented in the following:

1. **Send message to all:** The House Controller sends the `DataResponseTriggerMessage` to all other classes except the battery.
2. **Response Messages:** Each class determines the data (consumption, production or price respectively) for this step and sends a `DataMessageAnswerCurrent` message with that information back to the House Controller.
3. **House Controller receives messages:** The House Controller receives all messages and saves all the data as local variables. Those local variables are attributes in the House Controller (see Figure 11).
4. **House Controller sends message to battery:** Once all those `DataMessageAnswerCurrent` messages are received, the House Controller sends a message to the battery containing the `powerBalance` that the battery needs to calculate the values.
5. **Battery sends message back:** The battery sends back a message to the House Controller
6. **Strategy is called:** The House Controller now has all information it needs in this step. So it calls the `strategy`.
7. **Strategy:** The `strategy` determines with how much energy we should charge or discharge the battery and how much we should buy or sell in this step.
8. **Execute decisions from the strategy:**
   - **Second message to battery:** A message with the decision from the `strategy` gets send to the battery.
   - **"Buy/sell energy":** We multiply the energy we want to buy or sell with the current electricity price and add or subtract it to the `moneySaved`.
9. **End of step:** Local variables in the House Controller get reset and the step is finished.

4.1.2 Optimized strategy forecast

The optimized strategy needs knowledge about the production, consumption and energy prices in the future. For that, it asks each class for a forecast for a fixed time period in the future using the `DataResponseTriggerMessageForForecast`. That happens each day at midnight. Those forecasts are saved as local variables in the House Controller.

4.2 High- and Low-Level Description of Objects or Modules

The following section describes the core modules of the simulation model in detail, starting with the House Controller.

4.2.1 House Controller

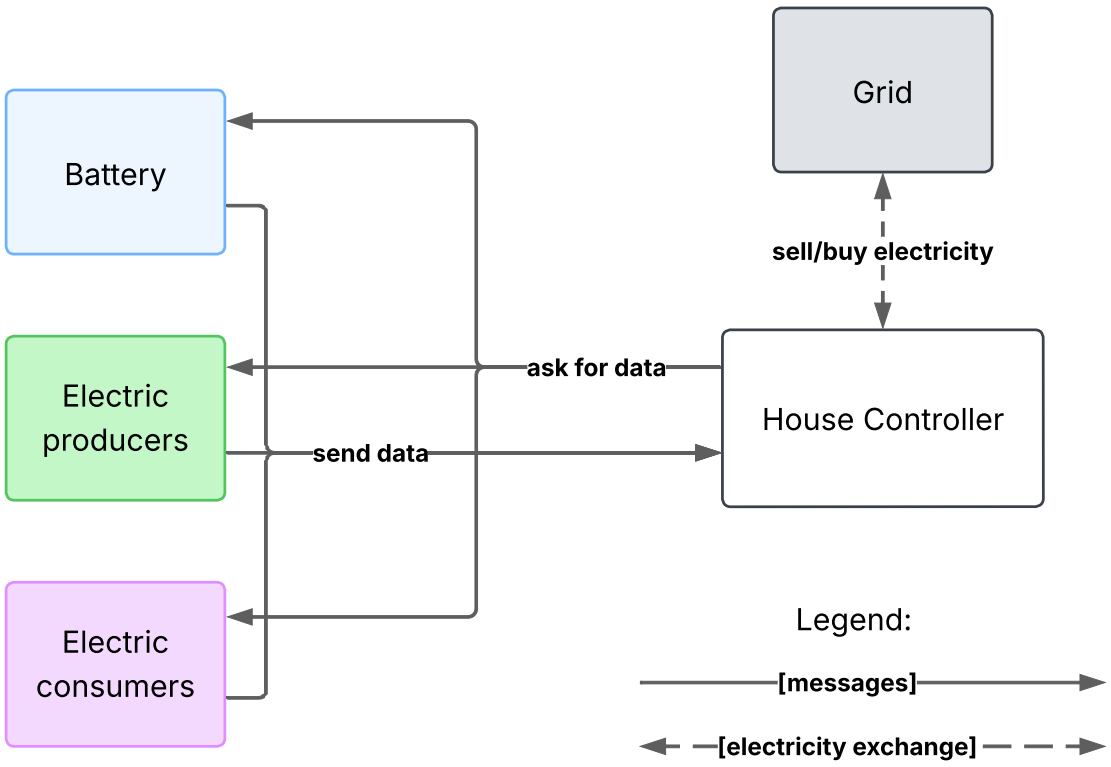

Figure 12: House Controller Diagram

The House Controller manages the whole simulation. Because of that, it has to interact with all other agents.

4.2.1.1 Messages The whole communication in the system works with AnyLogic Messages, which are sent through the ports in the agents. You can see here all message classes we use. They all inherit from the abstract "MessageType" class. You can see a visualization of these classes in Figure 13. It contains the two attributes `sender` and `timestamp`. The `sender` stands for the agent from which the message is sent and the `timestamp` is the time when the message is sent. All the classes have no members, just some have attributes which stand for the the data which is sent with the message. All time stamps in the messages are `double` values, which represents the time spent since the beginning of the simulation. They can easily be converted

to a date format whenever we need that.
The messages on the right of the dotted line are sent from the House Controller to the other classes:

- **DataResponseTriggerMessage**: Is sent each step from the House Controller to all producers, all consumers, and to the price class. It just triggers the response from those classes, so it does not have any attributes.
- **DataResponseTriggerMessageForForecast**: Is sent when forecast data is needed. This depends mainly on the optimized strategy (see Section 4.2.2). For this message, there are the `startTime` and the `endTime` for the time period for which the forecast should be sent back.
- **DataMessageToBattery**: Is sent to the battery each step. It does not work exactly like the messages to all other classes because the battery needs more information about this step to do its calculations. To be precise, it needs the `powerBalance`. That is also the reason why the message to the battery is sent last in each step.

The messages on the left of the dotted line in Figure 13 are messages that are sent bach to the House Controller as a response. There are three kinds of messages for that:

- **DataMessageAnswerCurrent**: This is the message that gets sent back to the House Controller at each step. Its further divided in producers, consumers and price, because each of them has a different types of quantity to send back. The House Controller has to distinguish electric prices from production from the PV for example.
- **DataMessageAnswerForecast**: This message gets sent to the House Controller as an answer to the `DataResponseTriggerMessageForForecast`. It sends a data array for each step in the requested time period. The differentiation between the three types is the same as in `DataMessageAnswerCurrent`.
- **DataMessageFromBattery**: For the battery things work a little differently. The message the battery sends to the House Controller, contains the `difference` and the `SoC`, which are explained in Section 4.2.4.

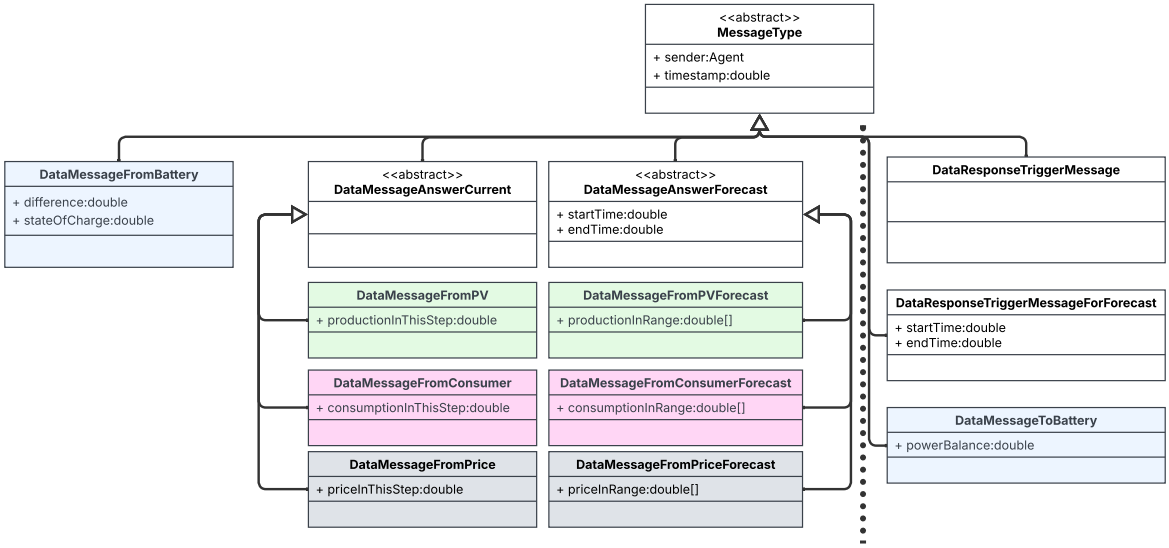

Figure 13: Message class Diagram

4.2.2 Strategies

To evaluate the performance of energy management in a household equipped with PV generation and a battery storage system, two distinct control strategies were implemented in the simulation model. These strategies reflect different levels of intelligence and forecasting capability in managing energy flows: a greedy strategy representing simple rule-based decision-making, and an optimizer-based strategy utilizing a mathematical model to plan energy use more efficiently. The comparison between the two provides insights into the potential benefits of advanced control systems for cost savings and the behavioral differences of the implemented classes.

4.2.2.1 Greedy Strategy The greedy strategy operates on a set of straightforward rules without any foresight into future energy prices or consumption. It purchases energy from the grid whenever local production and battery storage are insufficient, and it sells excess energy when the battery is full and generation exceeds demand. While easy to implement and commonly used in basic energy systems, this approach often leads to suboptimal economic outcomes and can in reality increase battery wear due to frequent charging and discharging cycles that are not strategically timed.

4.2.2.2 Optimization Strategy The optimizer-based strategy formulates energy management as a Mixed Integer Linear Program (MILP) that seeks to maximize economic efficiency over a forecast horizon. It incorporates future electricity prices, PV production forecasts, and household consumption patterns to make globally informed decisions about when to buy, store, or sell energy. The optimizer is updated daily and plans 1.5 days ahead, allowing for more efficient use of the battery and reducing unnecessary grid exchanges. This strategy assumes the presence of a smart energy management system and access to a functioning battery. Furthermore to reduce complexity in this limited simulation environment, the optimizer operates under the assumption of perfect foresight regarding consumption, generation, and price signals.
Let the forecast horizon be divided into T timesteps. The following variables and parameters are defined:

$T \in N$	: number of forecast timesteps
$c \in R^T$	: vector of electricity prices
$x \in R^T$	: vector of energy bought from the grid
$p \in R^T$	: net power from local generation and demand
$b \in R^T$	: battery state of charge (SoC)
$g \in R$	: maximum (dis)charging rate
$s \in R$	: initial battery SoC
$d \in R$	: degradation per energy cycle
$\mathrm{f} : R \to R$	: battery (dis)charging efficiency function
$b_{\text{init}} \in R$	: initial battery capacity (normalization)

The optimization problem is defined as follows:

Objective:

$$\min_{x} \; c^T x \tag{1}$$

Minimize the total cost of purchased energy over the planning horizon.

Constraint 1: Battery state of charge bounds

$$0 \leq s + \sum_{i=1}^{t} \mathrm{f}(x_i + p_i) \leq b_t \quad \forall t \in \{1, \ldots, T\} \tag{2}$$

Ensures the battery SoC remains within physical limits.

Constraint 2: Battery degradation

$$b_{t-1} - d\left(\frac{x_t + p_t}{b_{\text{init}}}\right) = b_t \quad \forall t \in \{1, \ldots, T\} \tag{3}$$

The battery capacity is recursively defined as the battery capacity of the previous step deteriorated by the rescaled change in state of charge in the current time step.

Constraint 3: Charge/discharge rate limits

$$-g \leq \mathrm{f}(x_t + p_t) \leq g \quad \forall t \in \{1, \ldots, T\} \tag{4}$$

Limits instantaneous charging/discharging power to the battery's rated capability.

Together, these constraints define the MILP:

$$\begin{aligned}
\text{minimize} \quad & c^T x \\
\text{subject to} \quad & 0 \leq s + \sum_{i=1}^{t} \mathrm{f}(x_i + p_i) \leq b_t && \forall t = 1, \ldots, T \\
& b_{t-1} - d\left(\frac{\mathrm{f}(x_t + p_t)}{b_{\text{init}}}\right) = b_t && \forall t = 1, \ldots, T \\
& -g \leq x_t + p_t \leq g && \forall t = 1, \ldots, T
\end{aligned}$$

While the system above explains the underlying constraints, its exact implementation is omitted here for readability. In practice, constraints 1 and 2 are combined, and additional Big-M constraints are introduced to accurately model the piecewise linear nature of the efficiency function f(). The complete mathematical formulation is available on GitHub [4].

Finally, after solving the system using the provided forecasts, the `HouseController` simply follows the calculated solution. In reality, however, forecasts are rarely perfect. If this assumption were relaxed, the controller would need to adapt its strategy to ensure not only feasibility, but also that the required perturbations remain as small as possible. Designing an appropriate heuristic for this scenario is a complex task and could not be implemented within the given time constraints. Nevertheless, several approaches — such as robust optimization or strategies that adapt dynamically during problematic timesteps — could be explored in future extensions of this project.

4.2.3 Database

4.2.3.1 Initial Architecture: The HSQL Approach

4.2.3.1.1 Concept and Objectives The primary requirement was to implement a robust database solution for managing simulation data within the AnyLogic environment. Various time-series data, such as photovoltaic (PV) generation profiles, heat pump load profiles, household consumption, and energy prices, needed to be stored persistently. The goal was to enable runtime-critical and flexible retrieval of this data via a dedicated API during simulation execution.

4.2.3.1.2 Decision for the HSQL Database Initially, the use of the native **HSQL database** provided by AnyLogic was evaluated. This solution was chosen for several strategic reasons:

- **Integrated Solution:** As the default database in AnyLogic, it eliminates additional setup or configuration overhead.
- **In-Memory Architecture:** The HSQL database operates in-memory, which allows for very fast read access—a critical advantage for time-sensitive queries during the simulation.
- **Simplified Interaction:** It uses standard SQL commands and provides a native Java library, which significantly simplifies direct interaction from within AnyLogic agents.

The decision was therefore made to leverage the existing and optimally integrated tools provided by the platform.

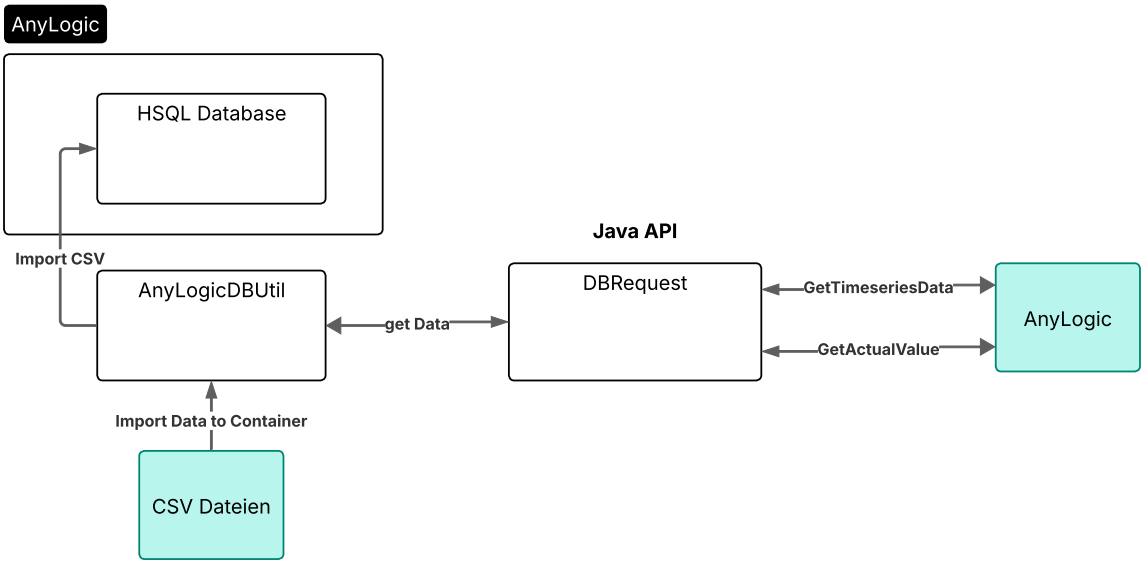

Figure 14: System architecture for data handling in AnyLogic.

4.2.3.1.3 Implementation Java and HSQL

Data Handling Utility (`AnyLogicDBUtil`)
The `AnyLogicDBUtil` class serves as the core database handler. It has two primary responsibilities:

1. **Data Import:** The `AnyLogicDBUtil` class was developed to fully automate the import process. Its main task is to sequentially read all CSV files located in the project folder and transfer them into the database. The key feature of this class is its **generalized approach**: it automatically detects column names and infers the appropriate data types from the headers and content of the CSV files. This eliminates the need for manual adjustments for each individual file, making the process highly efficient and scalable.

2. **Data Access:** It contains the logic to directly fetch data from the database via internal methods, represented by the `-get Data` call in the diagram.

Data Retrieval API (`DBRequest`)

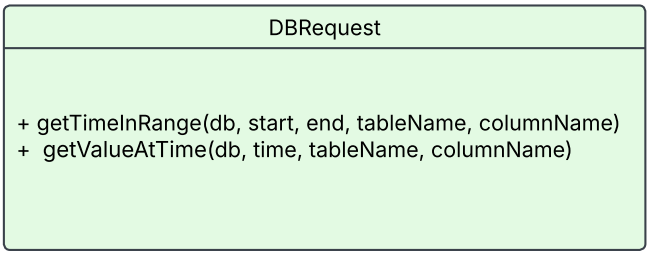

Figure 15: UML class diagram of the `DBRequest` class. It provides methods to query a database based on time, such as retrieving data within a specific range (`getTimeInRange`) or for a specific moment (`getActualTime`).

After the database is initialized by the `CSVImporter`, the `DBRequest` class serves as a central API (Application Programming Interface) for all other models and agents in the simulation. (See figure 15) It provides standardized functions to access the imported data. Two primary methods were implemented for this purpose:

- `getTimeInRange(db, start, end, tableName, columnName)`: This function enables the querying of an entire **time range**. It returns a collection of data points from a specified table and column for a defined start and end date.

- `getValueAtTime(db, time, tableName, columnName)`: This function is designed for **point-in-time queries**. It retrieves the exact data value that corresponds to the current simulation time (`time`). This is essential for providing models with the relevant value (e.g., the current energy price) at runtime.

4.2.3.2 Scalability Issues with HSQL

4.2.3.2.1 Prototyping with HSQL Database The initial implementation was first validated with smaller, synthetically generated test datasets, each comprising about 1,000 rows. In this controlled environment, the developed solution based on the HSQL database functioned flawlessly. Both the automated import of the CSV files and the critical runtime data retrieval via the Java API ran without problems, confirming the fundamental viability of the concept.

4.2.3.2.2 Confrontation with Real-World Performance and Scaling Problems

The crucial limitations of the chosen approach became apparent when the implementation was tested with a real-world dataset intended for a PV model. This dataset included a multi-year time series in 15-minute intervals, resulting in a CSV file with over 600,000 data points—a 600-fold increase in data volume compared to the initial tests.

When attempting to import this dataset, severe problems occurred. Although the import process did not crash completely, an inspection of the target table in the HSQL database revealed massive data corruption.

In numerous rows, error messages like `java.lang.ArrayIndexOutOfBoundsException` appeared instead of the expected values.

To isolate the source of the error, further tests were conducted:

- An import attempt using AnyLogic's native functionality (via an Excel file) led to the exact same error pattern.
- Using another large CSV file ruled out damage to the original file as the cause, as the errors remained identical.

These results made it clear that the problem was not in the custom Java code or the source files, but rather represented a fundamental limitation of the HSQL database in handling large volumes of data within the AnyLogic environment.

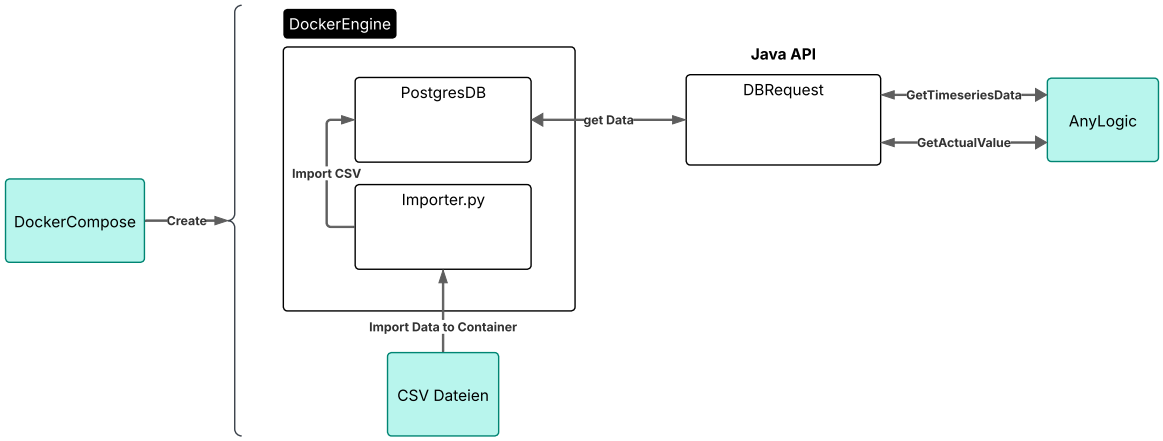

Figure 16: Overview of the revised database architecture using Docker and PostgreSQL

4.2.3.3 Pivot to a New Architecture: PostgreSQL with Docker

Due to the identified shortcomings, the decision was made to switch the database technology. This also opened up the possibility of using Python for data processing. Two alternatives were evaluated:

1. **Redis:** An in-memory database that, while extremely performant, proved to be very complex to implement for the specific retrieval of time-series data.
2. **PostgreSQL:** A robust, relational database. Although it is not purely "in-memory," it was favored due to its simpler implementation and the assumption that its performance would be sufficient for the simulation's requirements.

The decision was made in favor of **PostgreSQL**. Switching to Python offers significant advantages, especially through the use of the powerful `pandas` library.

4.2.3.3.1 Implementation of the new Architecture The final architecture consists of two main parts: a containerized backend managed by the **Docker Engine** and a **Java API** for the simulation model.

Dockerized Backend

A `docker-compose` command is used to create and manage the backend environment. This environment contains two key services running in containers:

- **PostgreSQL Database (`PostgresDB`)**: A robust, relational database that stores all the time-series data.
- **Python Importer (`Importer.py`)**: This script replaces the original Java-based importer. It follows the same principle of automating the import process but leverages the superior data handling capabilities of Python. Before loading the data, the script performs necessary preparation steps, such as handling date formats. The use of the `pandas` library makes this process significantly simpler and more robust, especially for large datasets.

The following example illustrates how the core logic for reading a CSV file and importing it into the SQL database can be implemented efficiently with just two lines of Python code.

```
# Efficient CSV import with Pandas
df = pd.read_csv(file_path, parse_dates=timestamp_columns)
df.to_sql(table_name, engine, if_exists='replace', index=False)
```

Listing 1: Efficient CSV import with Pandas

Java API and Simulation

For the simulation-side data retrieval, the previously implemented `DBRequest` class was largely reused. Therefore, the existing API, with minor adjustments to its database connection string, remained a valid and sufficiently performant solution for querying the new PostgreSQL database.

Results of the New PostgreSQL Database Architecture The data import now worked without issues, regardless of the size of the CSV files, and retrieval with DBRequest functioned smoothly. But there are some performance bottlenecks during the Simulation.

4.2.3.4 Identifying the Runtime Performance Bottleneck Until this point, the actual runtime performance could not be tested, as the simulation classes responsible for data retrieval were not yet finalized. The hope was that the query speed of the PostgreSQL solution would be sufficient for the existing hardware.

However, after integrating all components, initial test runs showed sobering results. The simulation ran at a speed of only about **13 simulated days per real-time second**. An extrapolation revealed that simulating a single year would optimistically require a computation time of about **40 hours** in the best-case scenario, even with five powerful computers running in parallel. This was unacceptably long for the practical use of the model.

The root cause analysis clearly identified the database queries as the bottleneck: Every 15 simulated minutes, **five models simultaneously sent a request** to the external PostgreSQL database. The repeated establishment of five parallel connections and waiting for responses drastically slowed down the simulation speed. Furthermore, since the future integration of an even more computationally intensive optimizer was planned, it was clear that this database implementation was not optimal for high-frequency runtime retrieval and would further reduce performance.

In the slow phase, the simulation theoretically sent **approximately 6,240 database queries per real-time second** to the PostgreSQL database.

4.2.3.4.1 Calculation in Detail Fundamentals

- **Number of models:** 5
- **Query interval:** Every 15 simulated minutes
- **Simulation speed:** 13 simulated days per real-time second

Calculation Steps

1. **Queries per simulated hour:**
   In one hour, there are 4 query points (60 min / 15 min).

$$4 \frac{\text{points}}{\text{hour}} \times 5 \,\text{models} = 20 \frac{\text{queries}}{\text{simulated hour}}$$

2. **Queries per simulated day:**
   A day has 24 hours.

$$20 \frac{\text{queries}}{\text{hour}} \times 24 \frac{\text{hours}}{\text{day}} = 480 \frac{\text{queries}}{\text{simulated day}}$$

3. **Queries per real-time second:**
   The simulation processes 13 simulated days in a single second of real time.

$$480 \frac{\text{queries}}{\text{day}} \times 13 \frac{\text{days}}{\text{second}} = \mathbf{6{,}240} \frac{\text{queries}}{\text{real-time second}}$$

This figure of over 6,000 requests per second makes it very clear why the simulation became extremely slow. Every single one of these requests generates network overhead and latency. The database thus became the absolute bottleneck, which perfectly underscores the necessity of the final in-memory caching solution.

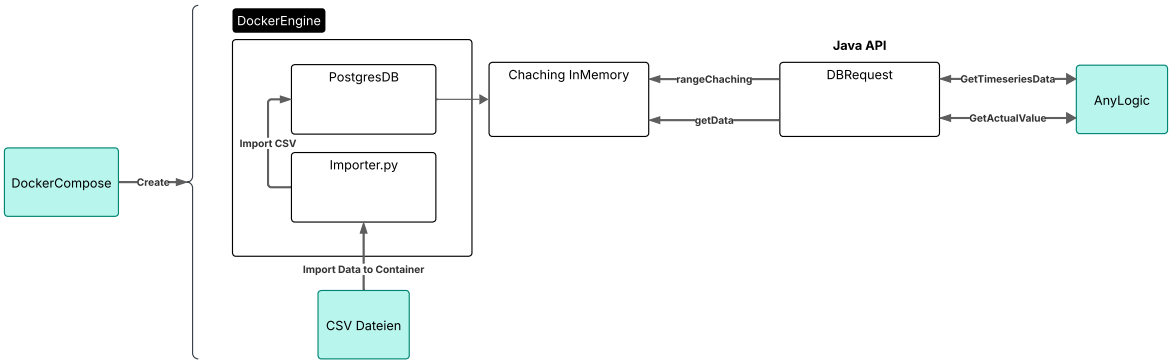

Figure 17: Overview of the revised database architecture using Docker and PostgreSQL

4.2.3.5 The Final Solution: High-Performance In-Memory Caching

To eliminate the latency of database queries, two approaches were evaluated: optimizing the SQL queries or implementing an in-memory data storage solution, similar to the original HSQL principle. The decision was made in favor of the **in-memory solution**, as it was faster to implement by extending the existing Java class (`DBRequest`).

4.2.3.5.1 Implementation of the Caching Mechanism

The core concept of the Implementation is to eliminate the high latency of database accesses (I/O operations) during the simulation's runtime. Instead of querying data from the database on-demand with each request, it is **proactively loaded entirely into main memory (in-memory) before the simulation starts**. This shifts the bottleneck from slow network and disk accesses to extremely fast memory accesses.
The implementation of this strategy is based on several key components:

Proactive Loading & In-Memory Cache A central function, `preloadTimeSeriesData(...)`, is executed once at the beginning of the simulation. It loads all required time-series data for the entire simulation period from the PostgreSQL database. This data is stored in a **cache**, which is implemented as a `ConcurrentHashMap`. This specific map structure was chosen to ensure **thread-safe and high-performance concurrent access** from the five simulation models without the need for manual synchronization.

Efficient Connection Management To optimize the initial loading process and avoid the cost of repeatedly establishing database connections, a **HikariCP Connection-Pool** is used. Instead of opening a new, expensive connection for each query, the pool manages a fixed number of reusable connections. This drastically reduces the overhead for communicating with the database.

Data Access in the Cache Once the data is in memory, all simulation queries (`getValueAtTime`) are performed directly on this local data structure. Instead of linearly searching through the potentially large list of timestamps, an **binary search** is employed. Since the data is sorted by timestamp during loading, the exact or nearest data point can be found in $O(\log n)$ time, which is practically instantaneous.

Through this combination of proactive caching, efficient connection pooling, and fast in-memory search algorithms, database queries during runtime are completely avoided, which significantly increases the simulation's performance.

4.2.3.5.2 Achieved Performance Increase and Runtime

This architecture led to a performance increase. The new simulation speed is now in an acceptable range of **1.5 to 4.5 simulated years per second**. A complete simulation run for one year now takes only about **5 seconds** without the optimizer. With the optimizer enabled, the runtime varies between **30 seconds and 6 minutes**, depending on the complexity of the problem. Thus, the performance bottleneck was successfully resolved, and the practical usability of the simulation was ensured.

4.2.3.5.3 Final Implementation and Integration in AnyLogic

The final solution is encapsulated in the Java utility class `DBRequest`, which serves as the central interface between the AnyLogic agents and the data logic. The interaction is divided into a one-time initialization phase and a continuous retrieval phase at runtime.
Initialization Phase: Proactive Loading of Data at Startup During an agent's startup sequence (e.g., in the constructor or the `on_startup()` method), all data required for the simulation is proactively loaded into memory. This pre-loading is the key to eliminating runtime latency. The process initializes a database connection pool and then calls the loading function, as shown in the following example from the constructor of a `PVJava` agent.

```
// Constructor of the PVJava agent
public PVJava(PV owner, double module_kWp, int module_count) {
    // 1. Initialize the database connection pool once
    DBRequest.initializeConnectionPool();

    // 2. Define the simulation time boundaries
    LocalDateTime simStart = LocalDateTime.of(2005, 1, 1, 0, 0);
    LocalDateTime simEnd = LocalDateTime.of(2023, 12, 31, 23, 59);

    // 3. Load the entire dataset from the database into the cache
    DBRequest.preloadTimeSeriesData(
        table_name,
        timestamp_column,
        data_column,
        simStart,
        simEnd
    );
    // ... further agent initializations
}
```

Listing 2: Preloading time series data into the in-memory cache on agent startup.

Runtime Phase: High-Speed Data Retrieval from the Cache

After initialization is complete, agents can retrieve data points at each time step of the running simulation. This operation now queries the in-memory cache instead of the external database, resulting in a query with virtually no latency. The following code snippet shows how a value for a specific point in time is retrieved from the cache.

```
// Method is called at each time step to get the current value
public double db_communication(double loopUpTime) {
    // Conversion of the simulation time into a standard LocalDateTime object
    LocalDateTime time = DateTimeConversionJava.doubleToCurrentLocalDateTime(
    loopUpTime);

    // Direct retrieval of the value from the in-memory cache
    double raw_kWh = DBRequest.getValueAtTime(
        table_name,
        timestamp_column,
        data_column,
        time
    );

    return raw_kWh;
}
```

Listing 3: Retrieval of a data point from the cache at a specific simulation time.

4.2.4 Battery

In the simulation, the class `BatteryJava` represents a household battery system designed to store surplus energy and supply it during periods of excess consumption. The battery interacts exclusively with the `HouseController`, which provides it with a power balance for each simulation timestep. This power balance is defined as the net energy flow in the household (production - consumption).

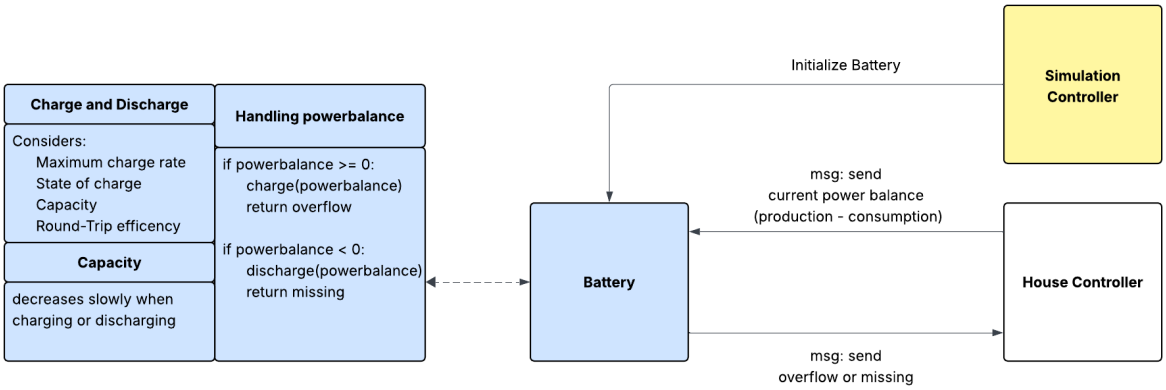

Figure 18: Basic overview of the Battery class

4.2.4.1 Parameters The battery model possesses internal parameters that include the current SoC, maximum capacity, current capacity, accumulated charge cycles, maximum charge and discharge power per timestep (in Kilowatt hour (kWh)), round-trip efficiency (a combined charging and discharging efficiency factor), and a degradation rate per full cycle.

4.2.4.2 Dis-/Charging When receiving a positive power balance, the battery attempts to store energy. It first scales the requested charge energy by the round-trip efficiency to account for internal losses. The actual energy that is stored is limited by both the remaining capacity and the maximum charge power. Any excess that cannot be stored is returned to the controller as overflow, indicating energy available for input into the grid. In contrast, when the power balance is negative, the battery attempts to discharge. If the `trueDischarge` flag is enabled, the model pre-compensates for round-trip losses by requesting more energy internally than externally needed. The discharge is constrained by the current SoC and the maximum discharge power. If the battery cannot provide the full amount, the remaining energy demand is returned to the controller as a "missing" value, to be met from external sources (e.g., the grid). Lastly the battery sends a response to the `HouseController`, which includes the net energy difference (either missing or overflow) and the updated SoC. This allows the controller to coordinate grid interaction or further optimize household energy usage.

4.2.4.3 Degradation The model tracks the number of partial and full charge/discharge cycles over time, updating the cycle count proportionally to the energy transferred. A degradation function is applied during each timestep to reduce the effective capacity based on the number of cycles, simulating the long-term aging of the battery by linearly approximating the real world degradation.

Overall, the battery serves as a dynamic energy buffer in the simulation, supporting self-consumption and minimizing grid dependence while incorporating physical and operational constraints such as efficiency, degradation, and power limits.

4.2.5 Photovoltaic

Photovoltaic modules are the key producers of clean energy for our simulated household and are thus vital for producing accurate results that provide helpful information to our user.
The seasonality of the PV modules also provides an interesting variance to the more static household energy consumption and correlates with the used wholesale prices due to a significant portion of electricity production being PV modules. In theory, this would lead to the necessity of buying batteries. General PV energy overproduction occurs when prices are low and therefore not saving this energy to prevent buying it later, when prices rise again, would cost us money.

First, a brief outlook of the system will be given with additional information later on. The figure below is a good starter to understand the PV system.

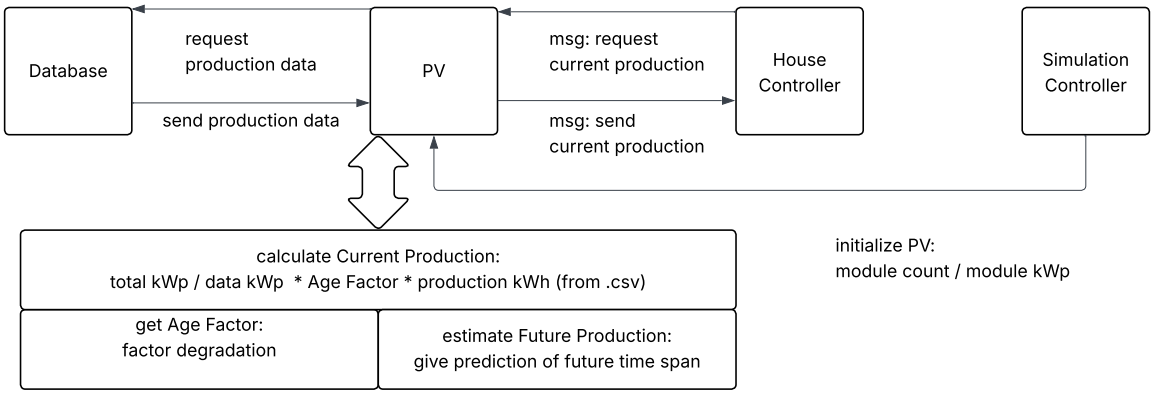

Figure 19: Basic overview of the PV class

There are 6 different functions in the PV class which are also present above:

- messageIn: Handles incoming messages from the House Controller. Depicted as the arrows between PV and House Controller.
- db_communication: Handles communication with the Database. Requests Data for a specific timestamp, depicted as arrows between PV and Database.
- PVJava: Constructor for the class, gets kWp per module and the module count, preloads data from Database.
- calculateCurrentProduction: Calculates the production for the current time step

$$\text{CurrentProduction}(t) = \text{base}_{kWh}(t) \times \frac{\text{total}_{kWp}}{\text{data}_{kWp}} \times \text{AgeFactor}(t)$$

$$\text{total}_{kWp} = \text{moduleCount} \times \text{module}_{kWp}$$

- getAgeFactor: Simple age factor of flat 0.8% yearly reduction of PV production
- forecastFutureProduction: Used to handle incoming messages that request future prediction responses.

Now, some specific methods will be discussed in more detail. Excluding messageIn because it just calls the corresponding methods and creates a simple response message and getAgeFactor since it only does the already mentioned very basic 0.8% yearly production reduction.

4.2.5.1 Constructor PVJava

The constructor PVJava does 2 things, first it gets the corresponding module kWp our PV modules use for the current simulation and also the amount of PV modules we want to simulate. In a previous version, it also got the size of the PV modules used as well as the roof size. This was then used to calculate the maximum amount of PV modules fitting on our roof. However, it was decided to move this function to the initialisation of the simulation instead. It is used there as an additional upper bound for the maximum count of PV modules that could be installed (besides the monetary restraints). This guaranteed us that the values stored in the simulation run index were the real amount of used PV modules, which simplified the cost calculation later on.
Note that all our classes were named xJava because our agents already had the basic x names. For example, the agent was named PV and the Java class PVJava.
The second responsibility of the constructor was the initialization of the database communication. Specifically, data gets preloaded for the entire time span covered by the corresponding csv file to accelerate DB interactions. In the case of the PV class, this time span ranged from January 1, 2005, to December 31, 2023.
Note that data before January 1, 2016, was not used, as our internal conversion method mapping the simulation time (in seconds since simulation start) to the date format used by the database uses January 1, 2016, as the default reference point for 0 seconds since simulation start. For further information, see 4.2.3.

4.2.5.2 db_communication

Whenever the programs needs to access one data point from a csv file, it calls the method db_communication and give it the corresponding timestamp in seconds since simulation start. First, we have to convert this time to the corresponding time format our Database uses for example: 1.1.2016. To achieve this, it uses the internal DateTimeConversion class. After getting the correct format, it calls the getValueAtTime function the DBRequest class provides. This function gets the correct table name, the column in which the timestamp is located, the column in which the data is located and the desired time. The b_communication method was developed pretty early and did not change since all improvements and changes only happened inside of the getValueAtTime method. To learn more about it, see 4.2.3.

4.2.5.3 calculateCurrentProduction

This is the core method used to calculate the energy produced. It gets the time we want to calculate and outputs the final kWh production for the given time slot. First, it calls the db_communication method to access the kWh saved in the csv file. With this raw value, some easy calculations are performed. There, it factors in both the kWp the source uses divided by the kWp the dataset uses, which is currently one kWp, as well as the 0.8% yearly degradation calculated by the getAgeFactor method. Below is the source code of the method:

```
public double calculateCurrentProduction(double lookUpTime)
{
    double base_kWh = db_communication(lookUpTime);

    double total_kWp = module_count * module_kWp;

    double kWp_Factor = total_kWp / data_kWp;

    return base_kWh * kWp_Factor * getAgeFactor(lookUpTime);
}
```

There were also ideas to extend this very basic implementation of PV production calculation by adding different angles or other factors. This however was not a high priority for our todo list due to this version already being functional and relatively accurate compared to real world data. Due to time constraints, those low-priority task could not be implemented.

4.2.5.4 forecastFutureProduction This method gets a range of seconds since simulation start, called start and end. It returns every time step in the given range including both the start and end time step. To achieve this, we first calculate the amount of steps wanted:

```
int steps = (int) ((end - start) / 900) + 1;
```

Now it initialises the double array that it returns with a length equal to the number of steps. Then it loops over every step in our steps-counter and calculate the timestamp which it wants to access by multiplying the current step it is at with 900 (= 15 minutes) and adding this to the start value:

```
for(int x = 0; x < steps; x++)
{
    double timestamp = start + x * 900.0;
    result[x] = calculateCurrentProduction(timestamp);
}
```

Here you can also see that the program uses the calculateCurrentProduction method for the forecast. This results in a perfect forecast which is not realistic. There were plans to extend the forecast and add some more realistic methods that could either add some randomness or use previous data saved to guess the future production. Due to time constraints, this more realistic approach could not be implemented.

4.2.6 Prices

The price class is a slimmer copy of the PV class, most interactions work the same and thus will not be repeated here. The messageIn and db_communication methods work the same way. There is no need for an age factor and the Constructor does not get any parameters from the Simulation Controller. The forecastProduction method also works the exact same way, the only difference is the getCurrentPrice method:

```
public double getCurrentPrice(double timestamp)
{
    double base_price = db_communication(timestamp);
    double price = ((base_price * 1.03) + 0.015) * 1.2;
    return price;
}
```

This is the implementation of the already mentioned contract[3] we use. The base wholesale price first gets modified with 1.03 and then you add a flat amount of 1.5 Cent per kWh and add the value added tax of 20%. Note that the german VAT tax rate for energy is 19% but the company has its origins in Austria and thus uses the local VAT tax rate for their demonstration of how the prices are calculated which were copied. The company also offers contracts in Germany.

4.2.7 Heat Pump

In the simulation the heat pump acts as an additional consumer in the house. At the beginning of the simulation it gets initialized by the Simulation Controller. In each simulation step it gets the request from the House Controller that it should send the current consumption back. The heat pump gets the data directly from the database. This data from the database is directly sent to the House Controller, without adjustment, because the data we use already fits to our specific house.
The forecast for the heat pump works very similar to the forecast for the PV-class (see Section 4.2.5). It gets a time period for which it returns an array containing the consumption for each simulation step in this time period. How we got the data we use for the heat pump is explained in Section 3.1.3.

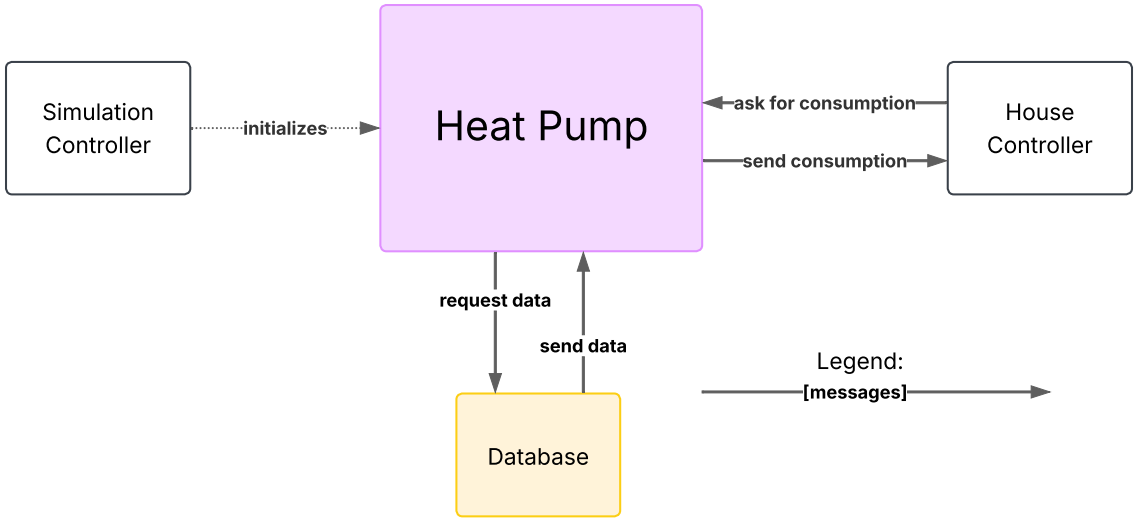

Figure 20: Heat Pump Diagram

[3] https://www.awattar.at/tariffs/hourly

4.2.8 Household Consumption

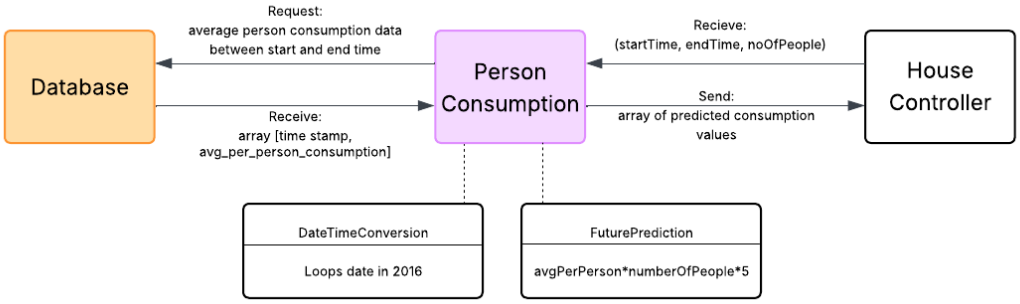

Figure 21: Person Consumption Diagram

The **Person Consumption** module is responsible for **calculating energy consumption per individual** in the household. It uses cumulative household energy readings from the database and dynamically scales the values based on:

- **Time intervals (15-minute slots)** – Energy used between two timestamps by taking the difference between cumulative values.
- **Number of people in the household** – Number of people living in the house to scale the load from a personal to a household level.
- **Scaling multiplier (5x)** – Adjusts raw per-person consumption values to align with realistic assumptions.

It is directly connected to:

- **House Controller** – Receives requests (current or forecast) and number of people.
- **Database** – Queries cumulative consumption values at specific timestamps.
- **DateTimeConversion** – Converts simulation time into real-world date formats for database queries.

Modules & Objects

- **House Controller (Caller)**
  - Sends start date, end date, and number of people to the Person Consumption class.
  - Triggers either current consumption calculation or forecast generation.
- **Person Energy (Core)**
  - Fields:
    - **int numberOfPeople:** Number of people in the house
    - **double multiplier:** Scaling factor (5x)
  - Responsibilities:
    - Stores inputs from HouseController.
    - Interacts with database and perform calculations.

- **Methods (Inside Person Energy)**
  - **messageIn(...):** Routes the incoming message (current or forecast).
  - **db_communication(time):** Queries database for cumulative household energy at a given timestamp.
  - **calculateCurrentConsumption:**

$$\text{avg\_per\_person\_consumption} \times \text{numberOfPeople} \times 5$$

  - **forecastFutureConsumption:**
    * Loops over each 15-minute interval between start and end timestamps.
    * Calls calculateCurrentConsumption for each step and stores results in an array.

Workflow

- House Controller sends start, end, and numberOfPeople to PersonEnergy.
- PersonEnergy stores the number of people and determines which method to run:
  - If current: call calculateCurrentConsumption
  - If forecast: call forecastFutureConsumption
- Each method uses db_communication to fetch cumulative data from the database
- Consumption is computed per interval
- Results are returned to House Controller, either as a single value (current) or an array (forecast).

4.2.9 Electric Vehicle

The Electric Vehicle (EV) estimates the daily energy consumption and charging behavior of a household EV using a parameter-driven approach. Since real-world EV driving or charging datasets were not available, the model relies on user-provided inputs such as annual kilometers driven and vehicle efficiency (in kWh per km) to compute energy demand. This energy demand is then evenly spread across a fixed 8-hour charging window, which corresponds to 32-time steps of 15 minutes each. Currently, the charging window is not user-defined and remains fixed; customization may be explored in future work.
The module interacts with the house controller to provide either the EV's current day energy demand or a simple forecast for a future period. However, the controller does not yet perform any dynamic optimization or scheduling of charging. This forecast can be used by the controller to align EV charging with available PV production, enabling smart load management and potential savings.
The EV behavior is implemented in a class named EVJava. It is capable of both real-time and forecasted energy reporting. The user specifies two main parameters:

- Annual driving distance in kilometers
- Energy consumption rate of the EV in kWh/km

From this, the model calculates the average daily energy demand as:

$$\text{Daily Energy Demand} = \frac{\text{Annual Kilometers}}{365} \times \text{kWh per km}$$

This daily energy demand is assumed to be consistent every day and is distributed over a predefined charging slot of 8 hours, which translates to 32-time steps (each of 15 minutes). For example, if a car requires 8 kWh per day, 0.25 kWh is assigned to each of the 32 intervals. The module handles two types of message triggers:

- For the current time, it returns the total energy used for that day.
- For forecasted periods, it returns an array of energy demands over time, filling only the early portion of each simulated day (first 8 hours).

Key simplifications made in this model include:

- The EV drives the same distance daily, regardless of day or season.
- The charging always happens in the same fixed window.
- The entire calculated energy demand is always charged within that window.
- No randomness or actual traffic pattern is considered.

Outputs from the module are sent to the house controller, which then integrates this demand into its broader optimization strategy. The following diagram explains the workflow of the Electric Vehicle EV

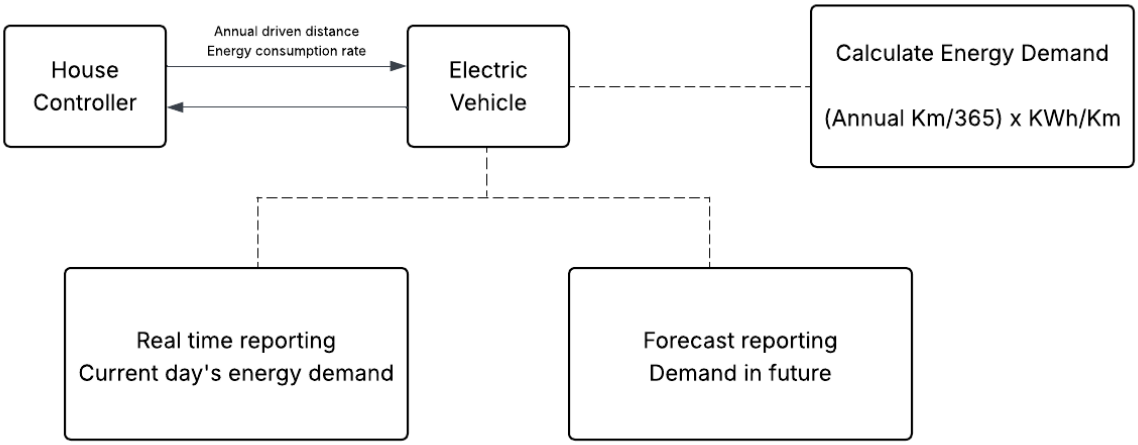

Figure 22: Electric Vehicle EV Diagram

4.3 Verification and Validation of the Model

4.3.1 Verification

The simulation has been extensively verified to ensure internal correctness. Particular attention was paid to the communication between components via message passing, which has been thoroughly tested in isolation and in integration. Each class and its behavior were validated under a variety of test scenarios to ensure expected responses and system coherence. Furthermore, the forecasting system has been explicitly checked to ensure that it returns accurate values for the respective forecast horizon at each time step. This guarantees that the optimizer operates on the correct input data at all times. It is also important to note that the current simulation introduces no randomness. Results are fully deterministic given the same inputs. This facilitates debugging and reproducibility of results.

4.3.2 Validation

Validation of the simulation results could only be carried out approximately, as a rigorous validation would require access to a very specific kind of dataset. While historical pricing data is readily available, the main challenge lies in obtaining a combined dataset of consumption and generation that originates from the same household under consistent conditions. Such a dataset would need to match our simulation parameters closely, with minimal deviations in hardware configuration, user behavior, geographic location, and time resolution. Additionally, detailed metadata about the setup and environment would be required to make a meaningful comparison. Despite this limitation, the overall results of the simulation align well with real-world expectations. In particular, the time it takes for most setups to become economically viable is consistent with reported values from similar real-world installations. One exception is battery performance, which appears slightly underestimated in the simulation compared to real-life conditions.

5 Simulation Results

5.1 Simulation Parameters

The main focus of our production run was to find the best setup for an average 3 person household while also reducing the resolution of our parameter space to the point where a full sweep was feasible. This consideration led to the following choice of parameters.

5.1.1 Battery

Most of the batteries parameters depend on the type of battery, in our case lithium ion batteries. Because of this, the chosen round trip efficiency is 0.98 and the maximal difference in SoC is 0.5 kWh for each timestep, regardless of the number of batteries. The degradation rate was selected such that the greedy strategy would lose 40% of its initial capacity over 8 years. Furthermore, to maintain a manageable size of the parameter space, the initial capacity per battery module was set to 2.5 kWh and the maximum number of modules purchasable was set at 5.

5.1.2 Photovoltaic

The technical information used for PV modules was obtained from an online retailer[2]. The exact values were used, except for the costs of the PV modules. This resulted in 0.44 kWp and an area of $2\,m^2$ per module. Note that, in order to reduce the parameter space, 2 modules were combined to one module unit. The cost of one kWp was derived by comparing various online sources. For the formula, see 5.2.1. Our roof size is exactly $100\,m^2$. This is used for calculating an upper bound of PV modules. The degradation was estimated to be an 0.8% yearly reduction in PV production.

5.1.3 Heat Pump

For the heat pump, we had fixed parameters. They are the ones we used to generated the heat pump data (see Section 3.1.3). Those values fitted to a normal household in Germany. That is what our simulation works with for the results.

5.1.4 Household Consumption

The simulation parameters for the Person Energy module aim to estimate individual energy consumption within a shared household, based on real-world data from Konstanz, Germany in 2016. Each simulation takes a start time, end time, and number of people as input, with all

timestamps internally looped to structurally equivalent points in 2016 due to the dataset's year limitation. The dataset provides cumulative energy readings at 15-minute intervals, so interval-based consumption is calculated by subtracting each reading from the previous one. This value is then divided by the number of occupants to estimate per-person usage, assuming uniform distribution. Since the dataset includes only shared household appliances and excludes personal devices like laptops or chargers, the captured data reflects roughly 20% of actual household usage. To scale it to a full-household level, we applied a multiplier of 5. This adjustment is based on the assumption that shared devices represent only a fifth of the total energy consumption. Additionally, because the dataset doesn't contain any information about individual users' energy use, we adopted a linear scaling method, which, while simplistic, allows the model to flexibly simulate varying household sizes despite data limitations.

5.2 Performance Measures

To assess the profitability of different household energy configurations, the simulation evaluates each setup based on a combination of investment costs and operational savings. Key variables include the number of installed PV modules, the size of the battery storage, and the control strategy employed (greedy or optimizer-based). The following sections present the mathematical formulation of the cost model, define performance metrics for financial evaluation.

5.2.1 Cost Model

The total investment cost for a given setup is composed of PV installation and battery costs:

$$
\begin{aligned}
c_{\text{PV}}(k) &= \begin{cases} 0, & \text{if } k = 0 \\ 900\,\frac{€}{\text{kWp}} \cdot k + 4500\,€, & \text{otherwise} \end{cases} \\
c_{\text{battery}}(h) &= 1000\,€ \cdot h \\
c_{\text{total}} &= c_{\text{PV}}(0.88\,\text{kWp} \cdot \text{pv\_count}) + c_{\text{battery}}(2.5\,\text{kWh} \cdot \text{battery\_count})
\end{aligned}
$$

This cost model reflects the parameters used in the final simulation run. Each PV module is assumed to have a capacity of 0.88 kWp, based on the choice of large-scale modules. While the installation of a PV system includes a significant fixed base cost, the installation cost for battery modules is dominated by the high unit price of the batteries themselves. As a result, the base installation cost for batteries is considered negligible and has not been explicitly modeled, in contrast to the fixed cost associated with PV installation.

5.2.2 Performance Metrics and Financial Normalization

The simulation evaluates the financial performance of each setup over the duration of the simulation (8 years) using several metrics.

Return on Investment

$$R_{\text{normalized}} = \frac{m}{c_{\text{total}}}$$

This metric is probably the most intuitive for most users. It simply represents the percentage of the initial investment that has been regained through operational savings. While straightforward, it does not account for differences in the expected lifespan of various components, which can lead to misleading comparisons between setups.

Value Gain Let m be the total savings on electricity, and let $d_{\text{battery}} = 0.4$ and $d_{\text{PV}} = 0.2$ represent depreciation factors:

$$\text{value gain} = m - (d_{\text{battery}} \cdot c_{\text{battery}} + d_{\text{PV}} \cdot c_{\text{PV}})$$

The value gain metric represents, as its name suggests, the amount of value accumulated through a setup. It assumes that the equipment itself has intrinsic value, which is gradually lost over the setup's lifespan due to depreciation. Notably, this metric tends to converge toward the total money saved minus the initial investment. A key advantage is that users can determine whether a setup has been profitable simply by looking at the sign of the result. Additionally, it allows users to compare setups against other investment options by evaluating their respective value gains.

Marginal Battery Benefit Let v_b denote savings with b batteries and v_0 the baseline without batteries:

$$\text{value gain per battery} = \frac{v_b - v_0}{b}$$

As seen in the analysis, batteries tend to perform worse than PV modules. For this reason, a metric that focuses specifically on battery performance can be useful. This metric expresses the value gain — as defined in the previous metric — on a per-battery basis, showing how much each battery contributes to the overall gain. It can also be interpreted as a measure of battery efficiency within a setup.

5.3 Simulation Results and Analysis

5.3.1 Electricity Cost Savings

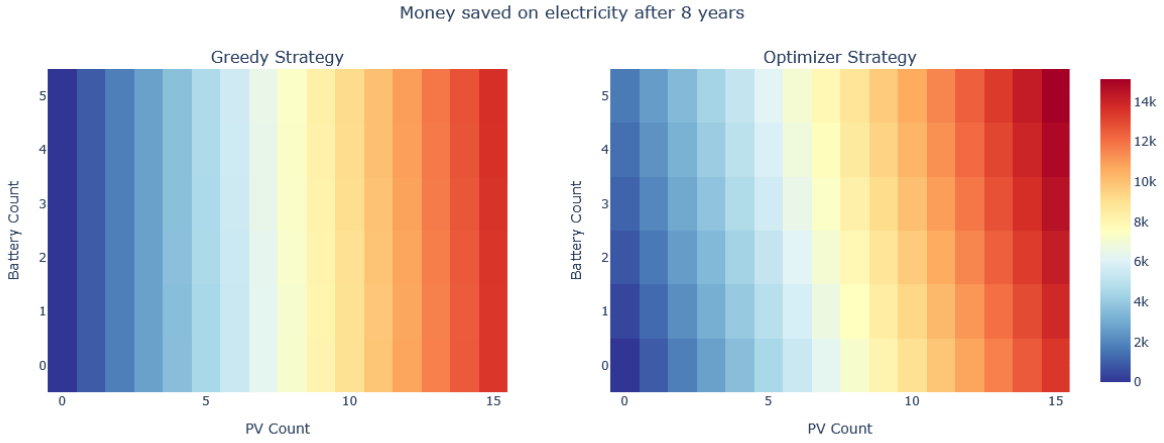

Figure 23: Electricity cost savings across configurations

(Figure 23) The first figure already offers valuable insights into the effectiveness of different configurations in terms of raw electricity savings. However, it does not yet account for the initial investment costs. As a result, setups that are significantly more expensive may appear disproportionately favorable in this metric, potentially leading to misleading conclusions when viewed in isolation.

5.3.2 Percentage of Investment Regained

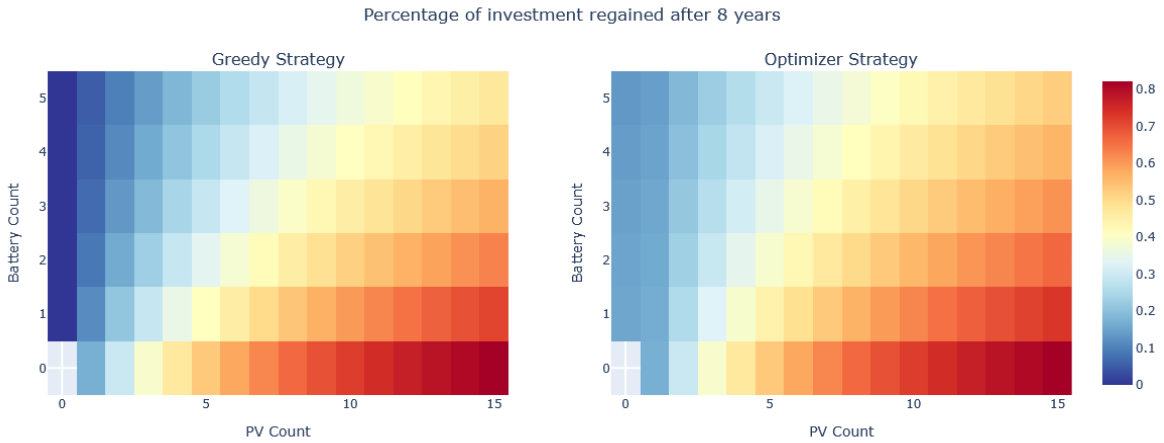

Figure 24: Normalized return on investment

(Figure 24) Another way to evaluate the setups is to examine how much of the initial investment has been regained over the duration of the simulation. This metric, often expressed as a normalized return on investment, is intuitive and allows for direct comparison between different configurations. However, it does not account for the varying lifespans of the individual system components. For example, a configuration consisting solely of battery storage would need to recover its cost within approximately 15 to 20 years, while PV modules typically remain operational for around 40 years.

5.3.3 Total Value Gained

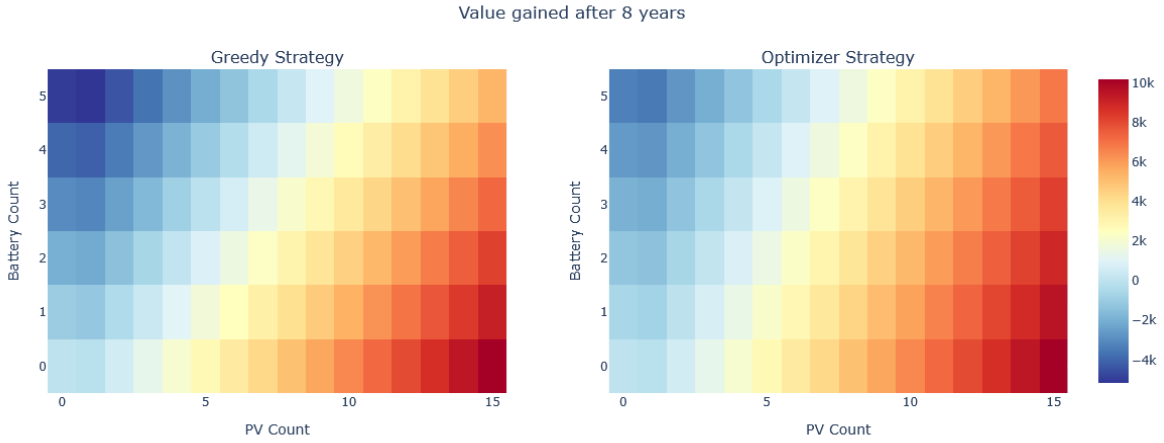

Figure 25: Net value gained after accounting for depreciation

(Figure 25) This figure uses the value gain metric, offering a clear view of the profitability of each configuration. As shown, under the assumptions made in this simulation, battery storage has a negative financial impact across all configurations. That is, in no scenario do the batteries pay for themselves. As suggested during the presentation, this result is likely due to the modeling choice of variable rather than fixed energy selling prices. It is also noteworthy that, although the optimizer strategy makes better use of the batteries than the greedy strategy, the gains are not sufficient to make batteries profitable. As a result, the most profitable configuration consists of the maximum number of PV modules without any batteries. Since the optimizer currently offers no advantage in setups without batteries, the best-performing configuration is identical for both strategies. However, this may change if additional flexible loads, such as an EV with controllable charging, were introduced. In that case, the optimizer could likely outperform the greedy approach even in battery-less configurations by better coordinating charging times with PV production and market prices. As explained in the definition of the metric, a positive value gain indicates that a setup has at least recovered the value lost through deterioration and generated additional value. Although batteries alone are not profitable, the combined setup may still yield a positive value gain when sufficiently many PV modules are included. It is also evident that setups consisting solely of PV modules become profitable only after the installation of at least two modules. This suggests that a single module cannot offset the fixed base installation cost over the 8-year simulation horizon. In summary, under the given assumptions and model parameters, the most profitable configuration is one with 15 PV modules, no batteries, and either strategy. This setup yields a value gain of approximately €10,000 over 8 years.

5.3.4 Value Gained per Battery

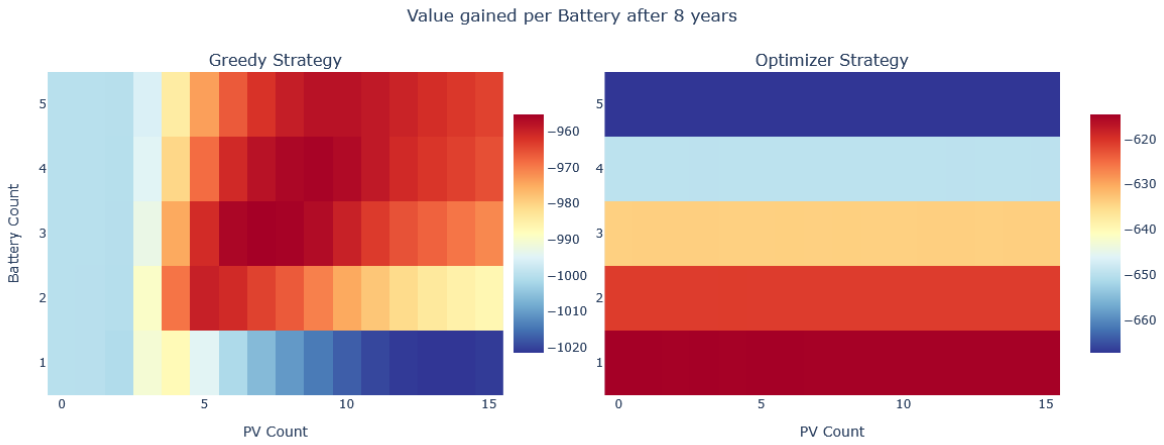

Figure 26: Marginal value gained per additional battery

(Figure 26) A closer look at the performance of battery storage systems reveals important dynamics within the simulation. As described in the corresponding metric, the plotted values represent the value gained per battery in each configuration. As seen in the previous figure, batteries reduce overall profitability in every tested scenario—even when used with the optimizer strategy. For the optimizer, battery efficiency appears to decrease monotonically with the number of batteries. In contrast, the behavior under the greedy strategy is more complex and warrants further examination. Not only is the efficiency trend non-linear, but it also lacks convexity. This suggests that, under different assumptions regarding battery costs, the most profitable configuration might not lie at the extremes of the parameter space (e.g., no batteries or maximum batteries). Instead, a global optimum might exist somewhere in between, and without a comprehensive sweep of the parameter space, we risk identifying only a local minimum. It is important to emphasize, however, that a higher value gain per battery does not necessarily imply a better overall setup. For instance, the highest gain per battery is observed in a configuration with 3 batteries and 7 PV modules, where the value gain per battery is approximately $-960\,€$. While this indicates the best relative battery performance, the total loss attributed to batteries in this setup is about $3 \times 960 = 2880\,€$. In contrast, a setup with 7 PV modules and only 1 battery shows a loss of around $-1000\,€$ per battery, resulting in a smaller total loss. This illustrates that even the most efficient battery usage (per unit) can still lead to significant overall financial losses. Ultimately, this metric provides valuable insight into the underlying behavior of the simulation and helps theorize how different parameter choices might affect performance. However, it is not directly suitable for most end users, as it does not reflect the overall profitability of a configuration in absolute terms.

6 Future Work

6.1 Technical Improvements

There are a few ideas for further work regarding our simulation project. We could utilize imperfect future predictions to get a more realistic House Controller knowledge base. Another approach would regard our optimizer which could work with wider optimization windows or factor in more complex optimization problems like non-linear battery degradation. We could also extend our simulation time span or simulate additional scenarios, either increasing our current variable granularity or adding additional ones like different inhabitant counts.
All those ideas would presuppose a significant improvement in runtime. This improvement is currently not realistic. However, NoSQL Databases could potentially be a useful approach to achieve this goal.

6.2 Model Extensions

There are also a few basic extensions to consider. The biggest one would be to enable the EV implementation. Currently it has no effect on the simulation, but with the needed data, we could factor in EV usage as discussed earlier. Additionally, we only envisioned uniform charging over a given time span of about 8 hours during usual sleeping schedules. This could be optimized to profit from price fluctuations during the mentioned time span. We could also extend the more basic implementations such as the PV production calculation with additional factors that have a real world impact on our simulated systems.

6.3 Assumption Revisions

Lastly, there are also certain assumptions that we could improve upon. Already mentioned was the modeling of selling energy. Another similar assumption is that our equipment used is from 2025 while all our data starts in 2016. We could either shift our data towards 2025 (which would be problematic due to the uncertainty of future energy prices) or shift the equipment used back to 2016. Another more general idea is to enable different household locations. Currently we only consider households from Konstanz. Updating PV data would be straightforward since we have an interactive map as our source, changing the personal consumption to a different city would be challenging, we would need to find a new and suitable source, our current source only contains Konstanz.

7 Conclusions

Our project was guided by the following research question: What is the optimal number of PV modules and batteries to maximize the financial benefit for a household?
Based on our findings, we conclude that purchasing batteries offers no direct financial benefit under the given assumptions. While batteries can provide independence from the power grid and potentially hedge against extreme future energy prices, our simulation suggests that, within our limited scope, investing in batteries is not advisable from a monetary perspective.
The situation is more complex considering PV modules. Although the investment does not fully pay off over the simulated time span of eight years, the results are still promising. An 80% return on investment within eight years and an estimated lifetime profit of approximately €10,000, accounting for the fact that the simulation only covers a fraction of the system lifetime, indicate that PV modules are indeed financially viable in the long term.
Additionally, since the fixed base cost of PV installation remains constant regardless of the number of modules, it is financially advantageous to install as many modules as possible to dilute these fixed costs. This further strengthens the case for maximizing the PV module investment.
Our findings should not be generalized without caution. Our tests indicated that most of the results strongly correlated with energy prices. Our energy prices started in the year 2016 before events like the Russian-Ukrainian war shaped our modern energy price landscape. We do not attempt to predict future energy prices especially not for the entire lifespan of a PV module, which can be more than 40 years. However, such predictions would be necessary to provide a future-proof assessment of the viability of batteries and PV modules.
There were also some other assumptions we made that have a major impact on our findings, especially our modeling of energy selling is not accurate to the real world, both the price you earn for selling energy and the availability of doing so is more disadvantageous in the real world. Selling excess energy becomes increasingly more important the more modules you install. A reduction in price and quantity of sold energy would likely increase effectiveness of batteries and reduce the effectiveness of PV overproduction.

8 Report Contribution

Table 1: Overview of author contributions to the report.

Section	Christian	Jakob	Nandini	Robert	Florian
1. Introduction	✗	✗	✗	✗	✓
2. Project Definition	✗	✗	✗	✗	✓
2.1.1 Requirements Definition	✗	✗	✗	✗	✓
2.1.2 Detailed Specification	✗	✓	✗	✗	✗
3. Input Modeling					
3.1 Data Collection					
3.1.1 Photovoltaic Production	✓	✗	✗	✗	✗
3.1.2 Electricity Price	✓	✗	✗	✗	✗
3.1.3 Heat Pump	✗	✓	✗	✗	✓
3.1.4 Household Consumption	✗	✗	✓	✗	✗
3.2 Data Fitting	✓	✗	✓	✗	✗
3.3 Validation of Input Models	✓	✗	✗	✗	✗
4. Model Description					
4.1 Conceptual Model	✗	✓	✗	✗	✗
4.2 Module Descriptions					
4.2.1 House Controller	✗	✓	✗	✗	✗
4.2.2 Strategies	✗	✗	✗	✓	✗
4.2.3 Database	✗	✗	✗	✗	✓
4.2.4 Battery	✗	✗	✗	✓	✗
4.2.5 Photovoltaic	✓	✗	✗	✗	✗
4.2.6 Prices	✓	✗	✗	✗	✗
4.2.7 Heat Pump	✗	✓	✗	✗	✓
4.2.8 Household Consumption	✗	✗	✓	✗	✗
4.2.9 Electric Vehicle	✓	✗	✓	✗	✗
4.3 Verification & Validation	✗	✗	✗	✓	✗
5. Simulation Results	✗	✗	✗	✓	✗
5.1 Simulation Parameters	✓	✓	✓	✓	✓
5.2 Performance Measures	✗	✗	✗	✓	✗
5.3 Results and Analysis	✗	✗	✗	✓	✗
6. Future Work	✓	✗	✓	✗	✗
7. Conclusions	✓	✗	✓	✗	✗

9 Production Run Example

In Figure 27 you can see an example of a run for one year. It is from 1. January 2018 to the 1. January 2019. The index is 189.

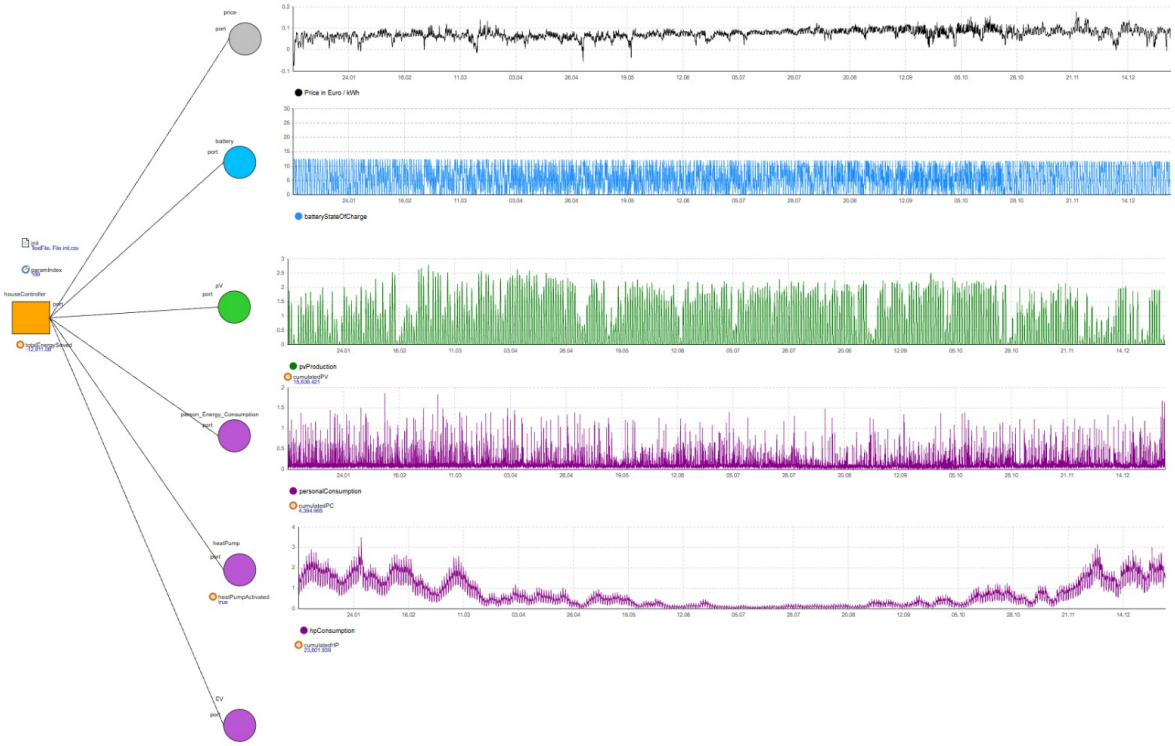

Figure 27: Screenshot of AnyLogic after simulation

References

[1] Martinez, Ana et al. PVGIS: Photovoltaic Geographical Information System. European Commission, Joint Research Centre (Ispra, Italy), 2025. Dataset collection JRC142887; Web-tool unter https://re.jrc.ec.europa.eu/pvgis/.

[2] SolarHandel24. Trina vertex s 440w bifazial glas-glas full black tsm-neg9rc.27 (staffelpreis), 2024. Zugriff am 27. Mai 2025. URL: https://solarhandel24.de/products/trina-vertex-s-440w-bifazial-glas-glas-full-black-tsm-neg9rc-27-staffelpreis?_pos=1&_psq=trina+440&_ss=e&_v=1.0.

[3] XplorodoX. Gantt chart with quarters. https://github.com/XplorodoX/SimulationAndModelling2Project/blob/main/FinalSubmission/Gantt_chart_with_quarters.pdf, 2024. Accessed: 2025-07-30.

[4] XplorodoX. Simulation and modelling 2 project. https://github.com/XplorodoX/SimulationAndModelling2Project, 2025. Last Visit at: 26. July 2025.